Akshay Adik

02-literature-review

Final Dissertation Submission

Document Details

Submission ID
trn:oid:::3117:628503132

Submission Date
Sep 23, 2026, 6:39 PM GMT+5:30

Download Date
Sep 23, 2026, 6:45 PM GMT+5:30

File Name
02-literature-review.pdf

File Size
362.0 KB

99 Pages

22,672 Words

125,181 Characters

*% detected as AI

AI detection includes the possibility of false positives. Although some text in this submission is likely AI generated, scores below the 20% threshold are not surfaced because they have a higher likelihood of false positives.

Caution: Review required.

It is essential to understand the limitations of AI detection before making decisions about a student's work. We encourage you to learn more about Turnitin's AI detection capabilities before using the tool.

Disclaimer

Our AI writing assessment is designed to help educators identify text that might be prepared by a generative AI tool. Our AI writing assessment may not always be accurate (it may misidentify writing that is likely human generated as AI generated and likely AI generated as human generated) so it should not be used as the sole basis for adverse actions against a student. It takes further scrutiny and human judgment in conjunction with an organization's application of its specific academic policies to determine whether any academic misconduct has occurred.

1 Literature Review

When production system deviates from its intended usage patterns or some unusual activity is detected during monitoring then system reliability, stability and security is under threat. These deviations in system behavior is called anomalies (Ahmad et al., 2025; Hagemann & Katsarou, 2020a; Sarda et al., 2023). These anomalies could be benign or malignant and details analysis has to be performed to identify and take precautionary measures. So, production support team which consist of Service Reliability Engineer (SRE) or On Call Engineers (OCE) conduct details root cause analysis of these anomalies. Typically, every anomaly is cause-and-effect relationship and can be grouped in different categories. These categorizations help engineers to perform the remediation steps as per the pattern. Anomalies can be grouped based on symptoms such as Performance, reliability or traffic anomalies. Another way to classify these anomalies are based on level of details or in which part of application layer does it exist. For example, Application level, service level or operational anomalies. Another way to classify is based on root cause of anomalies like resource system failures, network communication failure, application faults or security breach. Below table enlist anomalies and its corresponding symptoms.

Table 1
Anomalies Classification and Symptoms

Anomalies	Symptoms
Performance	Increased response time, latency, CPU leaks, and I/O bottlenecks (Fernando et al., 2025; D. Liu et al., 2021; Noor, n.d.; Sarda et al., 2023; Tien et al., 2019; Xing et al., 2025; Yu et al., 2021).
Reliability	Failure to perform a particular service operation (D. Liu et al., 2021; Sarda et al., 2023).
Traffic	An increase or decrease in service requests based on geography, time of day, or other contextual factors (D. Liu et al., 2021).
Application	Failure symptoms observed across the application as a whole (Soldani & Brogi, 2023). These can be further classified into performance, reliability, security, or other categories.
Service	Failure observed in a specific service or part of the services within the overall application (Soldani & Brogi, 2023).

Anomalies	Symptoms
Operational	Slow response time, increased error rates, and other operational symptoms (Sarda et al., 2023).
Resource/System Failure	CPU, memory, disk I/O, container, or node-related failures (Pei et al., 2025; Pham et al., 2024a; Rouf et al., 2024; Sarda et al., 2023; Zhou et al., 2019; Z. Zhu et al., 2024).
Network Communication	Network latency, delays, packet-related issues, connectivity problems, and similar failures (Fu et al., 2025; H. Huang et al., 2024; Lee et al., 2023a; Pei et al., 2025; Pham et al., 2024a; Z. Zhu et al., 2024).
Application Fault	Faults caused by complex microservice interactions, configuration issues, or software defects (C. Zhang, Peng, Sha, et al., 2022; Zhou et al., 2019).
Security	Security threats such as Denial-of-Service (DoS) or Distributed Denial-of-Service (DDoS) attacks (Fernando et al., 2025; Nobre et al., 2023b).

Anomalies can be introduced throughout the software development life cycle (SDLC). Some of the anomalies can be introduced in early stages of SDLC but can be found in later stages which highlights importance of testing and monitoring.

Table 2
Anomaly Origin in the Software Development Life Cycle (SDLC)

Fault Origin in SDLC	Anomalies
Design and Development	Runtime issues found in an application are often caused by incorrect implementation or flaws in the system design (Gulenko, 2020).
Deployment and Configuration	Incorrect configuration or deployment setups can degrade application performance, create security vulnerabilities, and cause service outages (Z. Chen et al., 2020; Sarda et al., 2023; Shetty, Bansal, Kumar, Rao, Nagappan, & Zimmermann, 2021).
Testing	Anomalies are typically injected during this stage to verify the stability and reliability of an application. Effective testing strategies can also uncover runtime issues introduced during the design and development phases (Sarda et al., 2023; Shivakumar, n.d.).

Fault Origin in SDLC	Anomalies
Operation	Unplanned downtime, performance issues, and operational failures can negatively affect application stability, reliability, and security (Z. Chen et al., 2020; Gulenko, 2020; Sun, Zheng, et al., 2025).

1.1 Historical evolution of Root Cause Analysis (RCA)

When a failure occurs in a production system, the team performs a structured review of the system to identify the cause of the failure. The goal is to perform the root cause analysis of the issue so that failures can be avoided in the future. To do this, SRE manually gathers information from logs, traces, or metrics and tries to understand how the incident started and its impact on the system over the period. They then apply structured analysis and determine the actual root cause of the issue (Y. Chen et al., 2023). Common techniques used in this process include “Five Why,” “Fishbone” or barrier analysis method. These methods are cost-effective and widely used; however, they require more time to identify the issue, need expert human input, and are error-prone in complex micro-service environments (S. Jin & Bei, n.d.).

Every production environment has a monitoring system in place that uses different rules to trigger alarms. For example, raise a alert if heap memory usage or CPU usage cross certain boundary These rules helps in identifying the problem which occurs repeatedly (Ding et al., 2023; Gan et al., 2023; Panwar et al., 2025; T. Wang & Qi, 2024). For example, MonitorRank uses a

heuristic algorithm that depends on a predefined view of the system's network topology to identify problems (Ma et al., 2020). This approach works well for simple issues; however, in modern micro-service environments, maintaining a large number of rules becomes impractical. Correlation-based methods also struggle with cause-and-effect relationships, which may lead to incorrect sources of failure. As a result, these approaches can be rigid and lack robustness, which may produce inaccurate results.

Anomaly Detection (AD) has evolved from statistical rule based system to advance machine learning or deep learning models. These models learn normal patterns and deviations. AD is the first step in the RCA. This is performed when the system encounters unusual patterns or behaviors (Jha et al., 2025). AD has evolved from using statistical checks, such as N-sigma (Pham et al., 2024a; C. Zhang, Peng, Zhou, et al., 2022) or clustering techniques such as BIRCH (Pham et al., 2024a) to modern deep learning models such as variational autoencoders (VAE) (Boesel et al., 2023; Z. He et al., 2023; Jha et al., 2025; Z. Zhu et al., 2024), LSTM or Transformers (Vervaet, 2023; S. Zhang et al., 2026). These models are trained offline on fault-free data to learn the normal patterns of the system (Z. Zhu et al., 2024). These models must refer to high-dimensional and nonlinear data and adapt to dynamic changes in micro-service environments. Variational autoencoders are used to construct anomalous patterns that reveal deviations; however, these techniques lack precision and accuracy in detecting anomalies.

Single model data approaches can be biased because they capture only part of the system state and shows the system behavior from one perspective. Multi-modal data gives complete system view and root cause analysis performed using this data has better fault detection rate compare to single modal data (Hwang et al., 2021; Yu et al., 2023). Frameworks such as CloudRCA, Eadro,

and MRCA integrate logs, traces, and metric data. This helps them to address broader range of faults (Lee et al., 2023a; Wang et al., 2024; Y. Zhang et al., 2021).

Hybrid RCA combines multiple data types and analytical techniques to identify root causes. It achieves this by building cross-layer reasoning (Hwang et al., 2021). For example, a CPU spike in the infrastructure layer increases the latency of the upstream system. Each data type brings in different perspective which helps in reducing missed anomalies and improve accuracy (Z. Xie et al., 2024).

1.2 Building Blocks of Root Cause Analysis (RCA)

Traditional root cause analysis involves anomaly detection, identifying the exact root cause, and fixing the root cause. These steps were conducted manually by an expert engineer. The latest developments in AI and DevOps have helped the RCA system analyze heterogeneous data and provide more plausible solutions using an automated pipeline (Cheng et al., 2023; Sarda et al., 2023; T. Wang & Qi, 2024). The main goal of a modern RCA system is to reduce the mean time to resolve (MTTR) (Sarda et al., 2023). The subsequent sections discuss the different stages of these pipelines.

1.2.1 Anomaly Detection (AD)

First, collect the data from from different sources, such as metrics, logs, or traces. Data can be structured, such as metrics, or semi-structured or unstructured, such as traces or logs. It perform normalization and aggregation steps to clean the data. The main challenges in this step are handling the huge volume of data along with its high velocity (Han et al., 2024; H. He et al., 2024; Wang et al., 2024; Zheng et al., 2024).

Preprocessed data are used by different machine learning or deep learning models to identify anomalous patterns or deviations (Pham et al., 2024a; Y. Zhang et al., 2021). Different sampling techniques are often used by these models during the training phase to identify deviations effectively (Ding et al., 2023; H. Huang et al., 2024). Models such as LSTM, VAE, and CNN have been used to detect anomalies (Jonnaikode, 2025; Pham et al., 2024b; Wang et al., 2024; Z. Zhu et al., 2024).

1.2.2 Root Cause Localization and Causal Inference (RCL/CI)

This is the core brain of the RCA system. The output of the AD phase is fed into these steps, which further preprocess and filter the data. The filtered data were used to detect cause-and-effect relationships, which helped to identify the root cause of the issue. Let us look at the process described below.

Data Preparation and Filtering. The system attempts to match alerts from the monitoring system with existing knowledge. System component matching alert are called Incident handlers. These handlers collect data specific to incidents from multiple sources. This information can be extensive, and different strategies, such as graph pruning and reinforcement learning (RL), can be used to isolate unwanted information from relevant information (Y. Chen et al., 2023). TraceDiag uses RL to reduce redundant components from the service dependency graph (Ding et al., 2023).

Causal Modeling and Root Cause Inference. Causal Modeling and Root Cause Inference. Data from the previous steps were used to create a graph of service interaction, which can form the basis for a cause-and-effect chain. Graphs are scanned to identify unusual patterns in different variables that cannot be explained by parents. This process is known as intervention recognition

(Hardt et al., 2024; Pham et al., 2024b; Z. Xie et al., 2024; S. Zhang et al., 2026). It uses different causal discovery algorithms, such as PC, Granger causality, and DAG-GNN (Pham et al., 2024c; T. Wang & Qi, 2024; Wang et al., 2024). KyLinRCA uses a cross-level modeling technique that connects the resource, container, and business layers (Jian, n.d.).

In the RCA using a graph-based approach, the services are treated as nodes and the communication link between the two services as edges. The system graph is modeled using this knowledge. Then, algorithms such as PageRank, Random Walk and Graph Neural Network traverse this system graph and rank the components that are most likely to cause failure based on dependencies (Pham et al., 2024b; Wang et al., 2024; Y. Zhu et al., 2024). Although they can identify the root cause, these approaches can be slow because of the unclear stopping point. The reinforcement learning method applied in TraceDiag and MRCA handles this by gathering enough evidence before stopping the analysis (Ding et al., 2023; Wang et al., 2024).

1.2.3 Interpretation, Explanation and Human Feedback

Result of root cause analysis should be easy to understand and engineer should be able to perform clear actionable steps. For example RCA Copilot and LasRCA group the incident into categories and provide clear explanation of those categories (Y. Chen et al., 2023; Shan et al., 2025). Large Language Model (LLM) can integrate the troubleshooting document and suggest actionable steps to resolve the issue (Han et al., 2024). In addition, engineer can interact with LLM models to understand the problem in much better way and improve accuracy (Cheng et al., 2023; L. Wang et al., 2023; Y. Zhang et al., 2021). Methods such as the Five Why can be used by SRE while interacting with LLM to extract objective evidence over subjective judgment. It helps in fixing common attribution errors and provides more accurate postmortem analysis (S. Jin & Bei, n.d.).

Modern micro-service-based systems generate huge amounts of logs, traces, and metrics per service. These data are unstructured, and the software does not understand these raw data. Deep learning can help in understanding raw data by converting unstructured data into a structured format. This is a type of teaching that helps computers read incidents. SoftNER can scan the entire logs and identify which service failed, what type of error occurred and shows the path of method execution. It uses an advanced neural network algorithm and identifies patterns from temporal data. This information is stored in a knowledge graph (T. Wang & Qi, 2024). DeepLog analyzes the log sequence to learn the normal patterns. When unusual activity occurs, DeepLog detects it and highlights which system has caused the issue (Girish & Rao, 2023; X. Ji et al., 2025; C. Wang et al., 2024; C. Zhang, Peng, Sha, et al., 2022). Deep learning methods are effective for identifying temporal patterns. They can detect certain signals that may be missed by simple heuristic rules. These models can produce clean and organized diagnostic evidence that can be used in postmortem reports and future learning (Z. Chen et al., 2022; Girish & Rao, 2023).

Once the model has identified the root cause of the defect, the next logical step is to use this information for future learning. Machine learning models such as KNN, Random Forest, Neural Network, and logistic regression can learn the classified patterns and predict the next classification based on the input. It can categorize the most affected services and show metrics that matter most (Fu et al., 2025; Nobre et al., 2023a; Zhou et al., 2019). For example, the BARO framework uses multivariate Bayesian online change point detection (BOCPD) and a robust scorer to analyze metric data to identify the main issues in micro-services (Pham et al., 2024b).

1.2.4 Auto-Remediation (AR) and Mitigation

An LLM can help generate remediation steps based on the root cause identified in the previous step (Shan et al., 2025). The LLM to historical incident data and the entire system context as an input while suggesting remediation steps. Deep reinforcement learning (DRL) and meta-learning can independently correct errors that occur during the learning process by their own (C. Ji & Luo, 2025a; H. Qiu et al., 2020). This helps in recovering and adapting quickly to changing conditions. These models continuously interact with the system and attempt to identify the best methods. This helps build a self-healing system that can detect issues, identify the root cause, generate a fix, and apply the fix without human intervention. Kepler and Amazon DevGuru use an RL-based approach to predict and apply corrective actions (C. Ji & Luo, 2025a). Reinforcement learning (RL) agents can restart pods, scale services, and change settings. They get rewards from metric services to learn the best configuration (H. Qiu et al., 2020). The FIRM framework uses a deep deterministic policy gradient algorithm to manage cloud resources better than traditional methods (H. Qiu et al., 2020). Meta-learning helps quickly adjust to unexpected failures (Chang et al., 2024).

ML Based resource management. Machine Learning models can predict resource adjustments rather than reacting to failure. ML models can predict system load changes and scale resources accordingly. The Narya system uses ML to decide between VM migration or reboot (Cheng et al., 2023), learning workload patterns instead of fixed thresholds.

Self Healing Systems. These systems are built using agentic AI. Multiple agents work independently and collaboratively to solve complex operational problems (Esposito et al., 2025). This

helps automate diagnosis, planning, and execution and reduces human burden (An et al., 2024; Pei et al., 2025; Shi et al., 2025) . For example, Nissist uses multiple LLM agents that understand the intent, retrieve the actions from the knowledge base, and generate a semi-automated remediation plan (An et al., 2024). FlowXpert enhances LLM agents with reinforcement learning to automate the troubleshooting workflow (Shi et al., 2025). These systems follow the reasoning and action (ReAct) pattern. This renders the troubleshooting process transparent (Cornacchia et al., 2025; Pei et al., 2025).

Automated Response. Once the problem and root cause are identified, the system can produce the output with the required code changes and configuration suggestions to optimize the resources or initiate recovery workflows. LLM help generate code and recover scripts or configuration files from the RCA output (Sarda et al., 2023). Prompt engineering can help generate a complete system code without human intervention. SynergyRCA generates a remediation command based on the identified root cause (Xiang et al., 2025). This helps in faster resolution of issues and reduces downtime and operational burden (Ding et al., 2023; Sarda et al., 2023).

Remediation Effectiveness. In this approach, the LLM is combined with a knowledge graph to extract the root cause of the failure. Learning happens from past events, current condition and operational data. To fix problems, a GenAI agent uses the Five Why method to ask questions about failures. Another agent examines reasons by comparing them to historical incidents (S. Jin & Bei, n.d.). This approach uses code and LLMs to identify underlying issues. It integrates cognitive processes with actions, resulting in reduced incidents over time (S. Jin & Bei, n.d.).

Knowledge Base (KB). Continuous improvement happens when the system learns from new data. It updates its knowledge and models. These models use the MAPE-K cycle: Monitor, Analyze, Plan, Execute, and Knowledge. The system watches, studies data, makes decisions, and saves information (Z. Chen et al., 2022; Esposito et al., 2025; Hrusto et al., 2022; Islam et al., 2025). The AIOPSLAB framework has validated agents by using fault injection and chaos engineering to simulate real-world failures (Y. Chen et al., 2025; Sarda et al., 2023). This helps identify whether agents can diagnose the problem with appropriate remediation strategies (Y. Chen et al., 2025).

Meta-Learning and Adaptation. Retraining improves performance, but it is often too slow when the system dynamically changes and evolves over time. Meta-learning helps model to adapt quickly to new failure patterns. Meta-learning techniques, such as Model-Agnostic Meta-Learning (MAML), allow models to generalize better (Duan, Bao, et al., 2024). It performs few-shot or zero-shot learning. This helps them learn new issues with little data. The performance of meta-learning is far better in environments where data are sparse or evolving. It helps in fast adaptation, sustains the diagnostic accuracy despite data drift, and eliminates issues in the long term (Duan, Bao, et al., 2024; Wang et al., 2025).

1.2.5 Three Pillar of Observability

It is clear from RCA literature that no single telemetry stream is enough to reconstruct the state of distributed system during the faults. This is frequently highlighted in literature but its reason is not reviewed to often. Metric, logs and traces capture partial view of the runtime state and success of the RCA system depends on how views of the system can be reconciled from observability into coherent picture (T. Wang & Qi, 2024; S. Zhang et al., 2023).

Treating these three pillars complementary is easy to justify since all of them shows different perspective of the same system. Metrics carry continuous numerical things like CPU usage, RAM usage, throughput and error rate. It helps to answer if something is happening and how badly it is impacting. Logs carry semi-structured text at runtime. It helps to answer and what and why of the system failures. Specifically when stack-traces or exception are emitted at runtime. Traces is only modality which encode the topology and temporal ordering across service boundaries. They help to answer where exact failure is occurring. System diagnosis in practice involves triangulating all three (Aktaş & Kilinc, 2024; Y. Chen et al., 2023; Cheng et al., 2023).

Suppose high traffic e-commerce deployment start returning HTTP 500s, The metric stream shows that latency and error rate is increasing. Log highlight the ConnectionPoolTimeoutException at persistence boundary. None of the signal is diagnostic in isolation since pool timeout can occur anywhere. But when trace data shows the order service stalled the connections for thirty seconds while inventory service respond normally. This give engineer some information to act upon. So, if any pillar is removed then diagnosis can collapse back into ambiguity. This is, I would argue, the structural reasons that empirical work keeps finding multi-modal pipelines outperforming single modality baseline on accuracy, robustness and false alarm rate (Lee et al., 2023a; Wang et al., 2024; Yu et al., 2023).

More recent development is that the tree canonical no longer enough to identify the failures within system. Code repository state, deployment events, configuration history, incident records, and even operator chat are increasingly being treated as legitimate inputs to RCA rather than as out-of-band metadata. The shift is conceptual as much as practical.

Why multi-modal data is necessary?. Three properties of multi-modal data occurs repeatedly across the literature. Multi-modal data improves the coverage because failure mechanism rarely manifest uniformly across modalities. Like slow memory leak is almost invisible in trace data. It improves the fidelity of the results by reducing the false positive rate which is high in single modality detectors (Huang et al., 2023; Wang et al., 2024). In addition it helps in improving the interpretability of the results. When LLM based layer is sitting on top of fused evidences, engineer can read it any time (Sarda et al., 2023; Sun, Luo, et al., 2025).

None of this is free and cost gets less attention that it deserves. Processing multi-modal data have trade-offs which matters materially. The first, multi-modal data fusion is complex. These data differ in structure, in temporal granularity and in cardinality. Representing them into single representation is modeling problem and this can create failure modes which did not exists in any of the input streams (Tao et al., 2024; Zhang et al., 2025).

The second issue is related to economics of velocity and volume. Micro-services produces huge telemetry data at rates that strain ingestion before performing analysis. Cost processing and storing the data outpace the marginal RCA value in return. This crossover is unclear and majority of time deployment specific (Y. Chen et al., 2023; Cheng et al., 2023).

The third is noise and incompleteness. Multi-modal pipeline is as reliable as its weakest input. Missing log levels, sampled-out traces or pre-aggregated metrics can quietly hide the anomalies the pipeline was built to surface (Fu et al., 2025; T. Wang & Qi, 2024).

The fourth and most underrated in the modeling literature is representational bias during training. Models exposed to multi-modal data tend to lean on whichever modality is most discriminative on the training distribution, and that becomes a liability the moment that modality is

degraded or absent in production. Feature-selection discipline and explicit modality balancing are therefore not stylistic refinements but design prerequisites (Tao et al., 2024).

A fifth constraint is labeling cost. Building labeled multi-modal data requires expert effort across all three pillars at once. In fast moving production environment, labeling pipeline rarely keeps up with the system it is meant to characterize (H. Huang et al., 2024; Lee et al., 2023a). This single constraint quietly shapes much of the methodological terrain covered in sections 2.4 and 2.6. It is the reason unsupervised and self-supervised approaches dominate the field. Same reason few-shot and in-context-learning methods have attracted the attention in the field.

So, multi-modal necessary but it does not justify itself. The diagnostic gains are real, and the empirical evidence supports them, but those gains are conditional on solving the fusion, cost, and labeling problems that single-modality RCA simply does not have to face. The rest of this chapter looks at how the field has tried to resolve those tensions, starting with the preprocessing pipelines that prepare the raw signals (section 2.4) and moving on to the modeling families that operate on the representations those pipelines produce (section 2.6).

1.3 Data Preprocessing

Data preprocessing is critical multi-stage requirement for effectively utilizing heterogeneous data. Raw data is noisy, voluminous, unstructured or misleading to get accurate diagnostic results. Data preprocessing often involves data cleaning and filtering, data transformation or feature extraction, normalization and time series alignment.

1.3.1 Data Cleaning and Filtering

Tables shows the raw data to be analyzed or transformed to improve the efficiency and accuracy of downstream models.

Table 3
Data Preprocessing Requirements and Techniques Across Observability Data Modalities

Data Modality	Requirement/Technique	Details and Purpose
General/Logs	**Filtering Irrelevant Data**	Irrelevant information should be removed to improve computational efficiency and reduce storage costs (Tang et al., 2025). This is particularly important in high-concurrency, high-volume production environments. Strategies may include filtering alerts raised fewer than a specified number of times per day and filtering logs based on time windows, location, or other relevant criteria (Chakraborty et al., 2023; Tang et al., 2025).

Data Modality	Requirement/Technique	Details and Purpose
Logs (Text)	**Noise Reduction**	Human-readable elements such as punctuation, newline characters, and unique identifiers including timestamps and IP addresses may be removed. Logs can also be converted to lowercase to reduce ambiguity (Sarika et al., 2023). Irrelevant patterns identified through regular expressions may also be removed (Cheng et al., 2023).
Metrics (KPI)	**Variance Filtering**	Since anomalies are typically reflected through fluctuations in metric values, metrics with low variance may be removed to reduce irrelevant features (Huang et al., 2023).

Data Modality	Requirement/Technique	Details and Purpose
Metrics (KPI)	**Handling Missing Values**	For categorical attributes, missing values can be replaced with a placeholder value (Bansal et al., 2020). For continuous attributes, missing values may be replaced using median values (Bansal et al., 2020) . Other approaches include imputing missing values based on patterns identified within the dataset (Almajed et al., 2025).

Data Modality	Requirement/Technique	Details and Purpose
Traces	**Data Cleaning**	Irrelevant or noisy data may be removed to support valid temporal analysis (Khodabandeh et al., 2025). Fields containing redundant information, such as `traceId`, may also be removed during transformation into tabular formats (Poghosyan et al., 2023).
KPI (Time Series)	**Edge Transition Removal**	Samples captured during the initial onset or transition phase of anomalies may be excluded because they can reflect unstable behavior and introduce noise into the analysis (Jonnaikode, n.d.).

1.3.2 Data Transformation and Feature Extraction

Transformation convert heterogeneous data into structured format. This include feature engineering to replaces important data with more valuable derived data. Below table describe transformation requirement and its purpose.

Table 4
Data Transformation requirement

Data Modality	Requirement/Technique	Details and Purpose
Logs (Unstructured)	**Template Extraction**	Unstructured log messages are converted into structured templates (Duan et al., n.d.; Duan, Bao, et al., 2024; Zheng et al., 2025). Algorithms such as Drain, IPLoM, and Spell are used for this purpose (Cheng et al., 2023; Duan et al., n.d.; Tang et al., 2025). If weak methods are employed, they can negatively affect subsequent anomaly detection (Duan et al., n.d.; Vervaet, 2023).

Data Modality	Requirement/Technique	Details and Purpose
Logs (Structured)	**Feature Extraction**	Structured logs are converted into numerical representations or vectors (Araujo et al., 2023; Sarika et al., 2023).
Logs (Structured)	**TF-IDF Encoding**	TF-IDF encoding generates log-frequency features to highlight distinctive words associated with errors for classification (Han et al., 2024; Sarika et al., 2023; T. Wang & Qi, 2024; Zhang et al., 2025; Zheng et al., 2025).

Data Modality	Requirement/Technique	Details and Purpose
Logs (Structured)	**Word/Sentence Embedding**	Methods such as GloVe, RoBERTa, and Semantic-BERT are used to generate feature vectors that capture the semantic meaning of log messages (Chakraborty et al., 2023; Sarika et al., 2023; C. Zhang, Peng, Sha, et al., 2022; S. Zhang et al., 2024). Log-tailored language models are designed to handle the characteristics of unstructured logs (H. He et al., 2024; Zheng et al., 2024).

Data Modality	Requirement/Technique	Details and Purpose
Logs/Metrics (Joint)	**Event Transformation**	Multimodal data is transformed into unified system events (Aktaş & Kilinc, 2024; T. Wang et al., 2024; Yao et al., 2024; Yu et al., 2023). Metrics can be used to generate events based on threshold rules or anomaly detection algorithms (Yao et al., 2024).
Metrics (Time Series)	**Feature Engineering (Time Domain)**	Statistical count-based, trend-based, and temporal features are extracted from time-series data (T. Huang et al., 2022). For instance, 12 types of time-series features can be derived to improve fault representation capabilities (Jian, n.d.).

Data Modality	Requirement/Technique	Details and Purpose
Metrics (Time Series)	**Feature Extraction (Frequency Domain)**	Frequency-domain characteristics are extracted using the Fast Fourier Transform (FFT) (T. Huang et al., 2022; Jonnaikode, n.d.).
Traces	**Feature Extraction (Trace Level)**	Traces are encoded into structural and status information (H. Huang et al., 2024). Key features include the latency of each service call, invocation relationships, and derived features such as the number of calls over a time interval and average duration (Han et al., 2024; Wang et al., 2025; S. Zhang et al., 2026).
Traces	**Feature Vector Construction**	Raw trace data is processed to obtain span vectors (Z. Zhu et al., 2024).

Data Modality	Requirement/Technique	Details and Purpose
General (Heterogeneous)	**Dimensionality Reduction**	Dimensionality is reduced to improve model performance (Xin et al., 2024). Techniques include Principal Component Analysis (PCA), Independent Component Analysis (ICA), and autoencoders (AE) (Xin et al., 2024) (ganSageUsingUn-supervised2021].
KPI (Time Series)	**Symbolic Aggregate Approximation (SAX)**	Time-series data is downsampled to create a more compact representation (W. Gu et al., 2025). Improved SAX representation (iSAX) incorporates trend variations to improve accuracy (W. Gu et al., 2025).

1.3.3 Normalization and Scaling

Normalization helps to analyze the data from different sources and scales without any bias toward features.

Table 5
Normalization and Scaling Requirement

Data Modality	Requirement/Technique	Details and Purpose
General	**Normalization**	Normalization filters and standardizes data to reduce bias (Gulenko, 2020). It also helps standardize data into a consistent format (Hwang et al., 2021).
Metrics (KPI)	**Min-Max Normalization**	Applied to standardize values into a unified range (Sarda et al., 2023). This technique is commonly used for numerical metrics (J. Chen et al., 2024; Jian, n.d.; Zhou et al., 2019).

Data Modality	Requirement/Technique	Details and Purpose
Metrics (KPI)	**Z-Score Normalization**	Standardizes data to have a mean of zero and a standard deviation of one. It converts metrics with different ranges into a unified scale (Jian, n.d.; Xin et al., 2024) .
Metrics (Raw Data)	**Unit Norm Scaling**	Raw metric data can be standardized by dividing each value by its range.
Metrics (High-Dimensional)	**Scaling for LLMs**	Time-series inputs are scaled to a limited number of significant figures. This reduces the amount of data required within the limited context window of an LLM (Y. Gu et al., 2025).

1.3.4 Time Series Alignment

Temporal alignment is crucial when high frequency metric logs are combined with low frequency traces. It helps in correlating cause-and-effect relationship.

Table 6
Temporal Alignment and Time-Series Processing Techniques Across Observability Data

Data Modality	Requirement/Technique	Details and Purpose
Metrics/Logs/Traces (Cross-Modal)	**Timestamp Unification/Synchronization**	Establishes a consistent time base across different data modalities (Tang et al., 2025).
Time Series (General)	**Temporal Alignment (Bucketing)**	Multimodal data is aggregated into a specified time granularity (Han et al., 2024). The time interval is partitioned into fixed-size buckets (Han et al., 2024; Hu et al., 2025; Tien et al., 2019).
Logs to Time Series	**Frequency Aggregation**	Log templates are converted into time series by counting the frequency of occurrences within defined time windows (Pham et al., 2024b; Wang et al., 2024; Zheng et al., 2024).

Data Modality	Requirement/Technique	Details and Purpose
Time Series (Anomaly Specific)	**Anomaly Alignment**	Time-series signals from the faulty node and candidate root-cause nodes are trimmed to their overlapping durations around the onset of the anomaly (Y. Lin, n.d.).
Metrics	**Sequence Decomposition**	To make time-series data more stable, it can be decomposed into trend and seasonal components using methods such as moving averages or Robust STL (Ge et al., 2024; W. Gu et al., 2025; Y. Zhang et al., 2021). This decomposition helps identify normal patterns and changes over time.

Essentially, the data preprocessing stage in contemporary RCA functions like a refining plant. Initially, the raw and diverse data (ore) must undergo cleaning (removal of impurities), followed by smelting (feature extraction and encoding) to form uniform components, and finally, it must be standardized (normalization and alignment) to ensure that the advanced machinery (ML/DL

and causal models) can accurately piece together the diagnosis (Araujo et al., 2023; Fu et al., 2025). It is very important to understand that mistakes in data preparation can harm the performance of best AI models. So, proper preparation is crucial to fully benefit from advanced AI systems (Duan et al., n.d.; Pham et al., 2024b).

1.4 Data Analysis

The analytical building block uses a machine learning-based approach. Instead of correlation or heuristic-based rules, machine learning along with graph-based or dynamic causal inference is used. Traditional RCA uses statistical-based analysis, such as Pearson correlation, rule-based approaches, or relies on simple heuristics, such as determining the root cause based on the earliest anomaly event.

Modern Root Cause Analysis (RCA) methods often use Causal Discovery. This involves creating graphs with techniques like the PC algorithm, Granger causality, and gradient-based methods, such as DAG-GNN in CausalRCA. More advanced methods include Graph Neural Networks (GNNs), Graph Attention Networks (GAT), and Hierarchical Reinforcement Learning (HRLHF) for complex reasoning and modeling dependencies (Gulenko, 2020; Jian, n.d.; Pham et al., 2024b; L. Wang et al., 2023; T. Wang & Qi, 2024; Wang et al., 2024; Zhao et al., 2025).

Relying only on correlation is not enough because “correlation does not ensure causation.” Simple graph methods often struggle with changes and confounding factors. Advanced causal methods solve these problems by looking at relationships over time or treating faults as interventions (Sometimes called Intervention Recognition). This helps find the root cause by identifying changes in the mechanism. Dynamic causal modeling is critical for modern micro-services because

system dependency graphs and causal mechanisms often change during an anomaly (e.g., due to congestion or load balancing).

1.5 AI Taxonomy

1.5.1 Foundation Capabilities and Constraints of RCA Systems

RCA system in a micro-services environment requires balance between cost, speed, scale, and explainability. Four key evaluation criteria are used, which reflect the main trade-offs engineers face while building an RCA system.

Efficiency and Deployability. The RCA system must operate quickly and inexpensively. They should not consume excessive computing resources or require long training times. Deployability is not only about speed or cost, but also includes (C. Chen et al., 2023). * How easy to integrate RCA systems * How difficult it is to maintain * How much operational efforts it requires once deployed

Expressiveness and Modeling Power. Micro-service systems produce complex and non-linear behaviors. The RCA must be sufficiently powerful to model these patterns. The trade-off is that higher modeling power comes with a higher computational cost and operational complexity.

Scalability and Robustness. In a real environment, data are noisy, incomplete, and continuously evolving. Service dependencies and patterns change over time, which is known as concept drift (Jian, n.d.; Panwar et al., 2025). RCA system must scale to * hundred or thousands of metrics * Large number of interconnected services * constant change in system behavior

Causality, Interpretability and Actionability. Engineers must understand and trust the results.- Causality: Understanding the actual cause of failures, not just what was correlated (Z. Chen et al., 2022; Harutyunyan et al., 2023). * Interpretability: Understand why the model flagged certain metrics or services as problematic. * Actionability: Detailed guidance on how to fix the issue and not just highlighting impacted services (Y. Lin, n.d.; C. Zhang et al., 2024).

1.5.2 Statistical Modeling Approaches to RCA

Statistical modeling has been used in the RCA for a long time. These models are efficient, require less computation, and are easy to deploy.

Strengths. Efficiency, Simplicity, Low Overhead. Traditional statistics use rule-based structures to highlight anomalies in production systems. These can operate with very low computational resources. Techniques such as the 3σ rule, ARIMA, vector auto-regression (VAR), Principal Component Analysis (PCA), and basic clustering require minimal training data (F. Chen, Zhang, et al., 2024; Cheng et al., 2023; Hagemann & Katsarou, 2020a). These approaches are extensively used by practitioners and are easy to implement. Owing to their widespread usage, these are easy to explain and maintain.

Bayesian methods, particularly Multivariate Bayesian Online Change Point Detection (BOCPD), represent an important advancement in statistical modeling. BOCPD model detects the changes in time-series data using probablistic dependency. BARO framework uses this technique and its performance is better than baseline statistical models. This shows that anomaly detection can use the statistical model with improved accuracy and computatinal efficiency (Pham et al., 2024a; Pham et al., 2024c). Due to its computational efficiency, these methods scale well with

large dataset. It also exposes the relationship among different parameters. This transparancy makes esier auditing and compliance for engineer and it improves the trust (Shan et al., 2025). As a result statistical analysis has become default choice in preliminary RCA (Fernando et al., 2025; J. Qiu et al., 2019).

Limitations. Expressiveness, Assumptions, Robustness. The statistical method assumes data follows common patterns over time (Almajed et al., 2025; H. He et al., 2024). In complex micro-service environments, nonlinear patterns violate this assumption (H. He et al., 2024). These methods show poor accuracy, with VAR achieving F1 score of 0.50 and AUC of 0.74, while PCA reaches F1 score of 0.56 for anomaly detection (Z. Chen et al., 2022; Fernando et al., 2025). They struggle with high-dimensional data and dynamic changes (Almajed et al., 2025; F. Chen, zhang, et al., 2024; H. He et al., 2024).

In micro-service environments, unrelated components may seem interconnected due to concurrent interactions, causing erroneous conclusions (Ding et al., 2023; Pham et al., 2024a; L. Wang et al., 2023). Simple rules like the 3-Sigma method often trigger false alarms, requiring manual adjustments (Harutyunyan et al., 2023). The Bayesian Online Change Point Detection (BOCPD) method aims to detect changes intelligently but is limited by data-based assumptions in dynamic environments (N. Chen et al., 2023). These methods can model basic patterns but may misidentify normal behaviors as anomalies. Modern cloud monitoring systems need precision, adaptability, and minimal manual intervention (Qi et al., 2022; Xing et al., 2025). Statistical causal discovery uses algorithms like PC, FCI, and Granger causality to create causal graphs (Cheng et al., 2023; Pham et al., 2024c; Pham et al., 2024a). Systems like CloudRanger and Microscope use these al-

gorithms and scoring methods for root cause analysis, but evaluations show performance similar to random selection (Pham et al., 2024a).

Statistical Causality and Graph-Based RCA. Engineers use correlation-based methods to check if Key Performance Indicators (KPIs) meet requirements (T. Wang & Qi, 2024). However, these methods often miss complex interrelationships between services, leading to false alarms (Ding et al., 2023). Statistical methods analyze metrics to establish cause-and-effect relationships by creating causal graphs based on timing and dependency patterns (Cheng et al., 2023; Pham et al., 2024c; Pham et al., 2024a). The monitoring system uses these techniques to identify the most likely root-cause service (Cheng et al., 2023; Pham et al., 2024a; Pham et al., 2024c). These systems show limited effectiveness, with accuracy similar to random selection (Pham et al., 2024c). Determining cause-and-effect from observational data requires substantial resources (C. Chen et al., 2023; Hardt et al., 2024), while micro-service systems' dynamic nature makes this process unreliable in complex environments (Gan et al., 2023).

Some statistical techniques, such as MonitorRank, use a graph-based approach to determine the root cause of a problem (Ma et al., 2020). In this approach, a service is considered a node, and a service call from one service to another is considered an edge. Then, they applied Random Walk or PageRank to simulate how the problem might spread through the system (Fu et al., 2025; Pham et al., 2024c; T. Wang & Qi, 2024). Some tools rank services based on how likely they are to be the source of an issue. These approaches do not require labeled dataset and are often fast and lightweight. They can be run in a production system (Fu et al., 2025). However, they struggle in micro-service environments, where service dependencies frequently change,and a simple graph traversal method does not capture the full complex dependencies.

In the process of constructing pathways, these algorithms frequently assess the current steps and determine the subsequent edge within the system. However, they can be misled by patterns that appear correlated but lack any causal relationship (Soldani & Brogi, 2023).

Bayesian Inference and Probabilistic Improvements. Bayesian methods use probabilities which helps them to manage uncertainty, such as weather spikes in response time. The Bayesian online change point detection (BOCPD) uses this approach and monitors time-series data to detect system behavior changes. This typically indicates the start of failure. It considers multiple metric changes over time to detect anomalies. It can estimate the exact time when a failure occurs (Pham et al., 2024a). If the anomaly detection is incorrect, the subsequent steps conducted to determine the root cause will also fail (Pham et al., 2024a).

Some RCA systems use clustering techniques with a simple machine learning approach to group similar metric patterns. Tools such as MicroRCA and MicroDiag use this technique. They group normal and abnormal behavior patterns together in different groups (D'Angelo & d'Aloisio, 2024; L. Wu et al., 2020; Pham et al., 2024a). These methods are easier to implement, cheaper, and beneficial for early stage RCA. However, their reliability in micro-service environments is still a concern compared to other modern approaches (Heeb et al., 2023).

The key performance indicator-based RCA system refers to many KPI's such as latency, error rate, and CPU usage, simultaneously. Adtributor and Hotspot detect failures more reliably when problems are not shown in a single KPI (C. Chen et al., 2023; Yu et al., 2023). However, this analysis incurs a cost. These systems must process large amounts of high-dimensional data. This creates memory and computational overhead on the system and increases the overall cost (C. Chen et al., 2023). The results of these methods are too high. For example, they may identify problems

in the service but do not clearly highlight the exact metrics or resources that caused the issue (Yu et al., 2023; Z. Zhu et al., 2024). Because of this lack of clarity in the root cause, SRE must manually analyze the logs, traces, and metrics to determine the real reason. This significantly reduces the practical value of these systems (Z. Zhu et al., 2024).

1.5.3 Classical Machine Learning Approaches

Capability Shift Beyond the Rules. ML-based techniques represent a significant shift from rule-based methods to learning from data. ML models analyze past and current data to identify normal and abnormal problems. Common ML algorithms used for this classification problem include isolation forests, logistic regression, support vector machines, decision trees, and random forests. These models help detect abnormalities and identify service interactions (Aktaş & Kilinc, 2024; Almajed et al., 2025; Araujo et al., 2023). Therefore, ML-based approaches learn from data, can detect small or early stage abnormalities, and understand service relationships and metrics better. It helps to detect the root cause much faster with better accuracy compared to rule-based systems (Gupta, 2019). Rule-based systems require experts to configure all the rules according to the evolving system; however, this part is automated by ML models, which do not require this tuning (Sarda et al., 2023). However, ML models are effective when trained on quality data (Gupta, 2019). ML techniques are grouped into supervised learning, which requires labeled data; unsupervised learning, which works without labeled data; and reinforcement learning, which improves through feedback.

Supervised Learning: Accuracy vs. Label Scarcity. The model was trained on normal and faulty system behaviors. Anomaly detection and root cause analysis are considered binary clas-

sification (yes/no) problems (Almajed et al., 2025; Gupta, 2019). Common model algorithms used in this approach include decision tree (DT), random forest (RF), support vector machine (SVM), and logistic regression (LR). When high-quality data are available, the performance of the supervised learning model is quite accurate. For example, decision trees achieve 89%-95% accuracy in detecting anomalies from log data. Support vector machines achieve nearly 98% accuracy with system call data (Aktaş & Kilinc, 2024; Almajed et al., 2025; Z. Chen et al., 2022). The MEPFL approach combines several models, such as K-nearest neighbor (KNN), random forest, and multilayer perceptron (Aktaş & Kilinc, 2024; Tien et al., 2019). These combined models were applied to distributed traces, achieving an accuracy of 89.1% (Nobre et al., 2023b).

The real challenge with the supervised learning approach is the unavailability of labeled dataset. In a micro-service environment, collecting a labeled dataset is costly. Human experts have to label the dataset, which is a time-consuming and practically unscalable approach (Cheng et al., 2023; Han et al., 2024; Y. Lin, n.d.). This may introduce bias or any other form of inconsistency due to human error (Cheng et al., 2023). In addition, fault data are rare compared to normal behavioral data. This creates a data imbalance or skewed data. Models trained on such data suffer from accuracy (Cheng et al., 2023). This makes supervised models costly and less feasible in large-scale micro-service environments. This is why real-world RCA systems prefer unsupervised anomaly detection methods (Fernando et al., 2025; Nikiforov, 2018; Xin et al., 2024).

Unsupervised Learning (UL). Scalability versus False Positives. This method primarily solves the problem of labeled dataset. Typically, models learn the data, identify patterns from the data, and flag anything that looks unusual or different (Araujo et al., 2023; Cheng et al., 2023; Fernando et al., 2025; H. He et al., 2024). Basically, it attempts to find data that do not look like

the rest (Xing et al., 2025). Common model algorithms used in this approach include isolation forest (IF), one-class SVM (OCSVM), K-nearest neighbor (KNN), local outlier factor (LOF), and clustering approaches such as DBSCAN (Aktaş & Kilinc, 2024; D'Angelo & d'Aloisio, 2024; Duan, Xue, et al., 2024; Fernando et al., 2025; Ge et al., 2024; Hagemann & Katsarou, 2020b; Petrov et al., 2022; Sun, Lin, et al., 2025; S. Wang et al., 2024; Xin et al., 2024). Among these, IF is effective in identifying 93% of anomalies with accuracy when used to automatically label micro-service data (Aktaş & Kilinc, 2024; W. Gu et al., 2025).

These methods create clusters by defining normal behavior by looking at the data and identifying the cluster where very few data points exist, which are flagged as anomalies (Panwar et al., 2025). This helps them classify and highlight unseen problems; however, without experts, these methods cannot classify which are harmless changes versus real failure. This may result in a high number of false alarms (Cheng et al., 2023; Han et al., 2024). Models such as OCSVM are sensitive to parameter configurations. Poor parameter selection can significantly reduce the accuracy and reliability of microservice environments (Ge et al., 2024).

The advantage of UL methods is their quick production deployment because they do not require a labeled dataset. However, their output needs to be validated by experts to rule out false alarms (Duan, Xue, et al., 2024; Fernando et al., 2025). In a micro-service environment, data are time series, and there is often a causal relationship in the data where order and timing play a role. UL models struggle with time-series data (Duan, Xue, et al., 2024; Ge et al., 2024; Vervaet, 2023). Principal component analysis (PCA) and distance/density-based methods fail to capture complex behavior patterns in dataset (Z. Chen et al., 2022; Duan, Xue, et al., 2024; Vervaet, 2023). In a dynamically changing micro-service environment, subtle or evolving anomalies are often missed, and normal changes may trigger false alarms (Duan, Xue, et al., 2024). Due to these limitations,

UL approaches are combined with advanced or hybrid approaches to better understand event timing and context (N. Chen et al., 2023).

Interpretability and Localization Gaps. ML-based approaches are far better than statistical rule-based approaches. ML-based approaches do not clearly explain decisions and do not identify the exact root cause. As a result, the engineer must investigate the root cause.

ML models act like a black box, where these models provide the output that the service is faulty but do not provide a clear explanation of how and why they have reached this decision (D'Angelo & d'Aloisio, 2024; Gan et al., 2022a; Poghosyan et al., 2023). These models provide service-level insights without much detail. Engineers must spend time debugging the issue to determine the root cause (Z. Chen et al., 2022; Y. Lin, n.d.; C. Zhang et al., 2024; S. Zhang et al., 2026).

The ML model can help identify whether the service is working as usual or whether any failure has occurred in the service. However, these models do not explain why the service failed or the exact problem. Engineers must search through logs, codes, metrics, or traces to identify the root cause. These models lack fine-grained details (Z. Chen et al., 2022; Harutyunyan et al., 2023; Wang et al., 2024; Zhang et al., 2024). SRE/OCE must trust the results, understand the reasoning, and act on the output quickly. Without proper explanation, confidence in ML-based RCA will be low, which will impact operational efficiency (Z. Chen et al., 2022; Harutyunyan et al., 2023). Because this research focuses on interpretability, final grained fault localization, and actionable explanation (W. Gu et al., 2025; T. Wang & Qi, 2024). This gap is addressed using deep learning models with built-in interpretability and Generative AI techniques.

1.5.4 Deep Learning for RCA

Representation Learning and Reconstruction-Based Models. Deep learning models learn the normal system behavior by analyzing the data. Once trained, these models can spot anomalies by identifying the extent to which new data deviate from what they have learned. Two widely used models are autoencoders (Aes) and Variational Autoencoders (VAEs).

An autoencoder (AE) is a neural network that consists of two parts: an encoder and a decoder. The encoder compresses the input data into a simpler form. The decoder reconstructs the original data from the compressed form (Hagemann & Katsarou, 2020a). The key idea is the reconstruction error, which signifies the difference between the original data and what the model has reconstructed (Gupta, 2019; Hagemann & Katsarou, 2020a; Hrusto et al., 2022; Y. Lin, n.d.). Autoencoders (AEs) are trained using data that is free from faults, enabling them to learn the system's normal operational behavior, metric dependencies, and inherent noise. Upon detection of abnormal behavior, the AE model attempts to reconstruct the data, resulting in a significant reconstruction error. This elevated error serves as an indicator of a potential anomaly within the system. AE models are much simpler than LSTM-based autoencoders and can be parallelized efficiently (Fernando et al., 2025). The reconstruction-based approach is robust when the training data contain a small amount of noise compared to the forecasting method (Z. Chen et al., 2022).

Variational Autoencoders (VAEs) extend the idea of autoencoders by learning the probabilistic representation of data over a fixed one. They capture the system data in the latent space, which captures uncertainty and variation (Y. Jin et al., 2025; Xing et al., 2025; Z. Zhu et al., 2024). VAEs detect anomalies by measuring the reconstruction error or reconstruction probability. The reconstruction probability indicates how likely the data are to be under the learned model (N. Chen

et al., 2023; Wang et al., 2024; Z. Zhu et al., 2024). VAEs use probabilities; therefore, they are better at handling changing system conditions, such as daily traffic spikes and seasonal demand. This helps reduce false alarms (C. Ji & Luo, 2025b; Wang et al., 2024; Z. Zhu et al., 2024). The system performs much better with VAEs than with Aes because they can model complex uncertain patterns in time-series data (Gao et al., 2025; Xing et al., 2025; Z. Zhu et al., 2024).

Autoencoders are commonly employed as preprocessing mechanisms, facilitating the extraction of significant features prior to the application of other Root Cause Analysis (RCA) techniques. Both Autoencoders (AE) and Variational Autoencoders (VAE) are integral to the RCA framework, such as the Multi-level Root Cause Analysis (MRCA). VAEs are utilized to rank anomalies, enhance subsequent stages of root cause identification, and mitigate the impact of early detection errors (Wang et al., 2024). These functionalities render AE and VAE indispensable for both anomaly detection and root cause analysis within large-scale micro-service environments (Gao et al., 2025).

Temporal Dependency Modeling (RNN/LSTM/GR). Recurrent neural networks (RNNs), along with their enhanced variants, Long Short-Term Memory (LSTM) and Gated Recurrent Units (GRU), are employed to identify anomalies in time series data, such as system logs and metrics (Adeba et al., 2025; Ge et al., 2024; Girish & Rao, 2023; Hagemann & Katsarou, 2020b; Y. Jin et al., 2025; Y. Liu et al., 2025; J. Qiu et al., 2019; Sun et al., 2024).

These models are designed to learn the sequence of events over time and predict subsequent occurrences. Discrepancies between the predicted and actual data are flagged as anomalies. Among these models, LSTM demonstrates superior performance due to its specialized gates, which enable the retention of information over extended periods (Girish & Rao, 2023; Hagemann & Kat-

sarou, 2020b; Jonnaikode, 2025; J. Qiu et al., 2019; Shetty, Bansal, Kumar, Rao, & Nagappan, 2021). This capability is utilized in root cause analysis systems such as DeepLog and LogAnomaly (Girish & Rao, 2023; X. Ji et al., 2025). In real deployment, the LSTM model fixed over 90% of the connectivity issues in offshore networks (Ahmad et al., 2025). The BLSTM achieved an F1 score of 0.717, which is better than that of the VAE and hidden Markov model (HMM), which showed strong performance in root cause analysis (Chang et al., 2024). RNN and LSTM are good at remembering past events, which helps them connect earlier system behavior with the current one. This is important for the RCA system, and that is why LSTM has become an integral part of hybrid RCA systems, such as LSTM-VAE and LSTM-Generative Adversarial Network (GAN), which detect anomalies across multiple related metrics (N. Chen et al., 2023; Ge et al., 2024; Jiang, 2024; Jonnaikode, 2025; Shivakumar, 2025).

Models process the data one step at a time, which makes them slower to train and run. In addition, the models are less suitable for real-time detection. This sequential processing blocks scalability and efficiency (N. Chen et al., 2023; Ge et al., 2024; Jonnaikode, 2025; Shivakumar, 2025). The new approach should aim to maintain the accuracy and improve the speed in the subsequent sections.

Parallel and Convolutional Models (CNN/TCN). Convolutional neural networks (CNN) and temporal convolutional networks (TCN) are effective in identifying patterns within time-series data, such as logs, particularly in detecting abrupt spikes. They encounter challenges with long-term trend analysis. CNNs operate by applying filters across data to extract local patterns, such as log events or performance fluctuations (Adeba et al., 2025; Gan, 2021; Hrusto et al., 2022; J. Qiu et al., 2019). Their parallel data processing capabilities render them more efficient than

recurrent neural networks (RNNs) (Jiang, 2024). When integrated with long short-term memory (LSTM) networks, CNNs enhance anomaly detection performance, achieving an F-score of 0.99 compared to 0.84 for CNNs alone (J. Qiu et al., 2019). Additionally, CNNs can extract features and reduce data dimensionality by transforming log keys into numerical representations (Adeba et al., 2025). TCNs employ dilated convolutions to examine long-term patterns without escalating computational demands (Xing et al., 2025). They train more rapidly than RNNs and overcome issues related to unstable gradients (Y. Jin et al., 2025). The TCN autoencoder requires 0.743 kiloseconds for training, in contrast to the 0.815 kiloseconds required by the variational autoencoder (VAE) (Chang et al., 2024). CNN models demonstrate high performance in real-world systems, achieving an accuracy of 94.99% when utilizing time-frequency spectrograms. The combination of CNN and LSTM networks enhances anomaly detection by capturing both local and long-term patterns (J. Qiu et al., 2019). CNNs and TCNs are particularly advantageous when rapid processing is essential (Adeba et al., 2025).

Global Dependency Modeling with Transformers. Transformer models capture relationships across entire sequences simultaneously, unlike LSTM's step-by-step approach. They identify long-distance relationships and early changes impacting later sequences (Hagemann & Katsarou, 2020b; Sun et al., 2024). These characteristics make transformers effective for multivariate time-series anomaly detection with complex metric interactions (C. Gao et al., 1 Aug.1, 2025; Ge et al., 2024).

The transformer handle data in parallel while using a multi-head attention method. This allows the model to analyze multiple relationships simultaneously. These models were trained faster and scaled better for real-time RCA systems. This is important for production-grade RCA systems (F. Chen, zhang, et al., 2024; Zheng et al., 2025). LogBERT and LogAnomaly use a

transformer-based approach to study log messages and identify related patterns across time. This helps identify meaningful patterns. These models converge faster than the LSTM-only approach (N. Chen et al., 2023).

The self-attention mechanism with a transformer checks how every data point relates to every other data point in the sequence. This helps the transformer learn long-range dependencies better than LSTM. These capture the relationship even if two events are far apart (Hagemann & Katsarou, 2020b; Huang et al., 2023). Attentions with transformers operate in parallel and avoid step-by-step processing (N. Chen et al., 2023; Ge et al., 2024; Huang et al., 2023). This makes them suitable for global temporal dependencies and feature interactions in multivariate time series data (F. Chen, zhang, et al., 2024; Gao et al., 2025; Ge et al., 2024).

Transformers reduce the training time and improve stability. RT-SemiVAE combines RNN with transformers and reaches stable training in 500 iterations compared to 800 iterations for LSTM-based approaches (N. Chen et al., 2023). HolisticRCA uses time-aware transformers with positional encoding and multi-head attention to better capture long-term metric behavior (Hou, 2025; Sun et al., 2024). Despite their strengths, standard attention mechanisms can struggle with complex repeating patterns in time-series data (F. Chen, Zhang, et al., 2024). To overcome this, some frameworks combine transformers with graph neural networks (GNN) to model temporal patterns and system dependencies (H. He et al., 2024). Transformers naturally provide explainability. Their attention weights show which part of the input sequence mattered most when making predictions. It helps to understand the model decision, trust the RCA output, and act faster during incidents. Therefore, transformers are widely used in systems attempting to achieve accuracy with interpretable anomaly detection results and maintain low computational costs (Sun et al., 2024).

Structural Dependency Learning with Graph Neural Networks (GNN). Modern micro-services have multiple services connected to each other in complex ways. These can be presented as a graph, where each service acts as a node and the communication path represents the edge (T. Wang & Qi, 2024). GNN are deep learning models designed to learn from such structures. Graph convolution network (GCN) and graph attention network (GAN) can model the propagation of faults or failures between services and detect abnormal behavior (Y. Jin et al., 2025; Somashekar et al., 2024; T. Wang & Qi, 2024).

GNN focuses on the relationship between services, whereas RNN, LSTM, CNN, and Transformer focus on metric changes over time. Each captures a different aspect of the system. The GNN helps to capture the origin of the problem, and temporal models detect when something unusual occurs. Both aspects are important for root cause analysis. Several RCA systems have successfully used GNN. CausalRCA learns causal dependencies using a DAG-GNN and then applies PageRank to rank the likely root causes (Pham et al., 2024a; Pham et al., 2024c). FaaSRCA uses a graph attention network (GAT) autoencoder on a global service call graph. It detects anomalies by identifying a part of the graph that cannot be reconstructed (Huang et al., 2024).

The GNN learns by passing messages between services that share information with their neighbors. This allows the models to learn the interaction pattern, detect abnormal behavior, and understand how the failure propagates. GNN work well with causal graphs that describe how failure propagates (Zhao et al., 2025). This mechanism naturally fits the complex, non-linear structure of micro-services (Somashekar et al., 2024; Zhao et al., 2025). Because they analyze the data sources together, GNN outperforms simple rule-based systems and manual analysis (Zhao et al., 2025). A recent approach combined a GNN with a transformer to capture structural dependencies using a

GNN and temporal behavior with transformers which leads to more accurate results (H. He et al., 2024).

1.5.5 Generative AI and LLM-Driven RCA

Rule Generation and Explainable Detection. Objective of root cause analysis is not only to detect anomalies but to explain why certain anomalies has occurred. GenAI address this challenge by formulating a rule that are interpretable and reproducible. GenAI models, such as ARGOS, employ LLM to construct anomaly detection logic as explicit rules based on opaque model predictions (Y. Gu et al., 2025). These rules function as intermediate, human-readable representations that effectively connect the processes of anomaly detection and root cause reasoning. Rather than classifying the data as normal or abnormal, LLM describe the anomalies in natural language and then convert those descriptions into executable Python code (Y. Gu et al., 2025). The generated rules should explicitly state what abnormal behavior looks like. Engineers should be able to inspect, validate, and trace anomalies to specific metrics or logs. The logic used for detection should be auditable and reproducible. (Y. Gu et al., 2025). This will significantly improve the interpretability of the root cause analysis.

ARGOS has shown significant empirical performance improvement where F1 score is improved by 28.3% over state-of-the-art methods on Microsoft dataset.Rule generated by LLM are found comparable to and sometimes better than human authored rules (Y. Gu et al., 2025). This indicates that explainable rule-based detection does not require a trade-off in accuracy, making it suitable for production systems.

Generative Sequence Modeling. GenAI models are trained on what normal behaviors look like over time. These models can then check whether new data follow a similar trend. These models predict what happens next, and if reality is different, they raise a warning. LLM-based methods operate in the detector mode, where the model examines current and past data points and predicts the next value or event in the time series (Y. Gu et al., 2025). If the actual outcome does not match this prediction, it is termed an anomaly (Y. Gu et al., 2025).

Monitoring data are obtained sequentially over time. These models treat this data as a sentence that needs to be continued. Models are trained on fault-free data so that they learn the usual order and progression of events or metric values. If the system produces an event that does not fit the learned pattern, the model flags it as abnormal. For RCA, this helps to understand when the system behavior first deviated from the normal pattern. For logs, these models work in a similar manner. For example, the LogGPT model predicts the next log message based on previous log entries. If the actual message deviates from model's expectation, it is identified as an anomaly (X. Ji et al., 2025). This methodology is useful in RCA because unexpected log sequences can reveal hidden faults, misconfigurations, or cascading failures.

GenAI models can detect faults early before major incidents occur (Y. Gu et al., 2025). This shift from reactive diagnosis to proactive prevention complements traditional detection stages and enables intervention before anomalies escalate.

LLMs for Reasoning, Explanation, and RCA Automation. Large Language Models (LLMs) are skilled at generating explanations in straightforward, human-readable language, which helps engineers in understanding the nature and cause of issues. These models independently scan diagnostic data to pinpoint the most likely root cause with detailed explanation (Y. Chen et al.,

2023). For example, RCACopilot can predict the root cause along with the reasoning behind the prediction (Y. Chen et al., 2023).

LLM do not require large amounts of data for training. Using few-shot learning, they can adapt to specific system domains by learning a small number of examples (Y. Chen et al., 2023; Shan et al., 2025; Zhang et al., 2024). This is useful when historical incident data are limited or specific to a particular system. This is contradictory to traditional models, which are black boxes and provide predictions without explanation (Shan et al., 2025; Sun, Luo, et al., 2025). LLM show reasoning steps in a structured and logical manner (Sun, Luo, et al., 2025). This approach enhance explainability (Poghosyan et al., 2023) and foster trust in the result (Sun, Luo, et al., 2025).

Large Language Models (LLMs) improve root cause diagnosis by explaining their conclusions which enables engineers to verify results (Sun, Luo, et al., 2025). In-context learning helps LLMs to incorporate new examples during analysis which reduces cost of retraining (Shan et al., 2025; Zhang et al., 2024).

By combining high accuracy, fast response time, and clear explanation, LLM-based RCA significantly reduces the time and expert efforts required to diagnose an incident (Y. Chen et al., 2023; Sun, Luo, et al., 2025). It is well-suited for real-time incident root cause analysis, helping teams resolve issues faster with greater confidence (Y. Chen et al., 2023; Xiang et al., 2025).

Hallucination-resilient multi-agent. GenAI models are powerful, but they have a known weakness of hallucination. This means that the model can produce answers that may sound correct but are actually incorrect. To make AI reliable for RCA, modern sources combine different external knowledge sources and cooperating AI agents.

Instead of relying on what LLM models remember from training, Retrieval-Augmented Generation (RAG) allows models to retrieve verified information from specialized databases such as run books, historical incidents, or system documentation (Pei et al., 2025; Shan et al., 2025; T. Wang & Qi, 2024; W. Zhang et al., 2024). RAG reduces hallucinations and improves accuracy by referring to real, historical, and verifiable data (Shan et al., 2025; Sun, Zheng, et al., 2025). Advance GenAI framework use multiple AI agents each with specific role. These agents work together to cross-check the results and improve the decision quality. For example, the Flow-of-Action framework uses standard operating procedures (SOPs) to guide agent behavior and embed SRE best practices into the system (Pei et al., 2025). This ensures that the analysis follows the proven operational logic. MABC employs blockchain technology for voting purposes, wherein agents assess outputs to reach a consensus on reliable results (W. Zhang et al., 2024). Graph frameworks such as MetaGraph and StateGraph integrate LLM with graph databases to model system relationships, thereby facilitating the prediction of cascading failures (Xiang et al., 2025).

In multi-agent systems, distinct agents are designated to manage specific tasks, such as CodeAgent for code analysis and JudgeAgent for validation (Pei et al., 2025). This specialization reduces the dependence on single model. This reduces the hallucination impact and improves robustness. By combining external knowledge with coordinated, specialized multi-agent systems, the GenAI framework produces more accurate, explainable, and verifiable results. This will reduce the error rate and make RCA much faster and more dependable (Pei et al., 2025; Shan et al., 2025; W. Zhang et al., 2024).

1.5.6 Multi-modal Data Fusion and Hybrid Intelligence

Fusion Strategies (Early, Intermediate, Late). Micro-service systems in modern cloud environments produce many types of data, such as logs, traces, metrics, and topological data. Each of these data points presents one part of the system behavior. On its own, no single data source is sufficient to fully explain why a failure occurs. Multimodal data fusion solves this problem, leading to a more accurate and reliable RCA.

In early feature-level fusion, raw data from different sources are merged and analyzed together (Lee et al., 2023b; Yu et al., 2023; S. Zhang et al., 2023) Systems such as Nezha and DiagFusion combine metrics, logs, and traces into a single unified representation, such as an event graph or shared embedding space (Yu et al., 2023). Using this, Nezha achieved an accuracy of 89.77. In this case, learning occurs across all data types, which helps the system discover relationships between traces, metrics, and logs. Simultaneously, combining the data into a unified format may lose or hide some essential data characteristics (Tao et al., 2024).

In representation-level fusion, each data type is processed separately to extract important features, which are then combined (Lee et al., 2023b; Tao et al., 2024; S. Zhang et al., 2026). Eadro and HolisticRCA follow this method. They model behavior within individual services and interactions across services while preserving the unique characteristics of metrics, logs, and traces. It retains modality-specific information, allowing integration across data sources. However, it makes the system architecture more complex than early fusion (Tao et al., 2024).

In late decision-level fusion, each data type is analyzed independently. Different methods that suit the respective data are used. The final root cause decision is made by combining individual results through voting or weighted scoring (Tao et al., 2024; Yu et al., 2023). For example, Pdiag-

nose identifies the root cause separately from the metrics and logs and then fuses the outcomes to produce the final diagnosis (Yu et al., 2023). This keeps the data type fully independent and easier to implement; however, it loses the deeper relationships between data types (Tao et al., 2024).

Studies consistently shown that using multiple data type together produce better RCA results than relying on single source (Lee et al., 2023b; S. Xie et al., 2026; Yu et al., 2023). Systems such as TrioXpert, which integrate multimodal data with LLM, have shown improved speed and accuracy of diagnosis in real-world environments (Sun, Luo, et al., 2025).

Causal Graph Construction from Multi-modal Signals. The advanced RCA system synthesizes a comprehensive system view by integrating data into causal graphs that illustrate problem propagation. MULAN transforms log data into a time-series format utilizing LLM to facilitate analysis alongside metric data. It employs contrastive learning to detect abnormal behaviors and utilizes KPI-aware attention to highlight key indicators. MULAN integrates metrics and logs to construct causal graphs that elucidate the manner in which anomalies precipitate system failures (Zheng et al., 2024).

TVDiag focuses on building specific diagnostic tasks within multi-modal environments. Its causal learning method connects metrics and logs to understand failures. Causal graphs from multi-modal data make diagnosis more explainable and actionable (S. Xie et al., 2026; Zheng et al., 2024).

Hybrid Learning and Transfer Learning Approaches. Hybrid approaches improve root cause analysis by combining different learning methods. Instead of relying on one algorithm, a hybrid system combines models that are good at understanding a specific aspect of system behavior,

as follows.- CNN are effective in identifying spatial patterns, such as correlations in metrics or components. - LSTM is good at capturing temporal patterns over time. - GNN models are effective in identifying the relationship between services and call graphs. - CNN + LSTM captures both spatial and time-based behaviors. - The transformer + GNN captures long-range temporal patterns with a service-level structure.

These hybrid combinations achieve better accuracy than a single diagnostic model (H. He et al., 2024; J. Qiu et al., 2019).

Training the RCA model from scratch is costly and slow. Transfer learning addresses this issue by allowing models trained on one system to adapt to similar systems. For example, MAD-DoC is an unsupervised transfer learning approach. This clusters similar storage components and transfers the learned knowledge between them. This helps reduce the cost by 60-70% while maintaining or improving accuracy (Boesel et al., 2023). Similarly, the cluster model (CM) approach groups similar services and shares models within each group. This reduces the number of models by two-thirds and reduces computational and resource costs (Fernando et al., 2025). For large-scale micro-service environments, more accurate, scalable, and cost-effective approaches are required, and together, these approaches work efficiently while maintaining high diagnostic quality (Boesel et al., 2023; Fernando et al., 2025; H. He et al., 2024; J. Qiu et al., 2019).

1.5.7 AIOps as an End-to-End RCA Paradigm

Automated Data Ingestion. AIOPS systems automatically collect data from micro-services systems. The data include metrics, logs, traces, and topological data. Different failures behave differently, and the system reacts accordingly. For example, RCACopilot uses a special-

ized incident handler for different types, which ensures that each problem is analyzed using a suitable diagnostic strategy (Y. Chen et al., 2023). Similarly, CloudRCA combines logs, KPI's and service topology to gain a holistic system view (Y. Zhang et al., 2021). Eadro and TrioXpert analyze within-service behavior and cross service dependencies avoiding incorrect conclusions derived from single modal data (Lee et al., 2023b; Sun, Luo, et al., 2025). Rather than collecting everything, AIOPS intelligently collects data. ML models determine which signals are most important for RCA, which reduces storage and computational costs without losing accuracy (Y. Chen et al., 2023).

Intelligent Diagnosis Pipeline. Once data are collected, AIOPS coordinates multiple diagnostic techniques to identify the root cause.- ML, DL, and causal inference methods build dependency, topology, and causal graphs. These graphs show how services and metrics influence each other (H. Wang et al., 2022; T. Wang & Qi, 2024; Zhao et al., 2025).

- Graph-based methods, such as TraceDiag, remove unnecessary components from service graphs using reinforcement learning, which simplifies diagnosis (Ding et al., 2023). BALANCE, another graph-based technique, groups anomalies into understandable categories, such as database errors and container faults, and explains why the cause is ranked highly (C. Chen et al., 2023).
- Causal discovery techniques, such as Granger causality and the PC algorithm, identify cause-and-effect relationships between metrics and services. Ranking algorithms, such as PageRank, highlight the most likely root causes (Pham et al., 2024a; Shan et al., 2025; Wang et al., 2024).

Instead of manual investigation, these methods produce a list of likely root causes, which reduces the Mean Time To Resolve (MTTR). This also helps reduce reliance on human expertise (Y. Chen et al., 2023; Ding et al., 2023; Soldani & Brogi, 2023).

Human-readable explanation. Finding the root cause is not enough. It must be understandable and actionable by the user. GenAI and LLM help produce complex outputs with clear explanations. LLM analyze diagnostic evidence, identify root cause categories, and explain their reasoning in human-understandable language (Y. Chen et al., 2023; S. Jin et al., 2024). SynergyRCA uses an LLM with RAG to produce RCA and recommend fixes (Xiang et al., 2025).

GenAI can automatically perform Five Why analysis. Knowledge graphs and live system data can be used to trace failures to underlying causes, such as coding mistakes or missing automation (S. Jin et al., 2024). This explanation turns technical RCA results into clear, actionable guidance, enabling faster remediation and decision-making (C. Chen et al., 2023; Shan et al., 2025; T. Wang & Qi, 2024). This aligns with the AIOPS goal of automated detection and resolution (Sarda et al., 2023).

Operational Impact: Reducing MTTR. The real value of AIOPS is the reduction in MTTR. SynergyRCA identifies the root cause in approximately 2 min (Xiang et al., 2025). RCA-Copilot achieves an accuracy of 0.7666 and enables fast and reliable diagnosis (Y. Chen et al., 2023). Higher accuracy means fewer false alarms and less time wasted on incorrect hypotheses. Some systems go beyond diagnosis and enable automated remediation processes. For example, the RCA output can be converted into ansible playbooks for direct execution (Esposito et al., 2025; Sarda et al., 2023). Transfer learning reduces the cost of maintaining RCA models across hundreds

of services (Boesel et al., 2023; Fernando et al., 2025). Multi-agent systems and voting mechanisms increase confidence in results and make automated actions safer (Pei et al., 2025; W. Zhang et al., 2024).

1.6 Economics

Organizations invest in RCA systems to reduce financial and reputational losses due to service downtime. The impact can be significant. For example, a Facebook outage in 2021 caused by a software defect resulted in an estimated loss of $ 60 million, while 37 minutes of downtime at YouTube led to approximately $ 1.7 million in lost advertising revenue (Sun, Luo, et al., 2025). Losses can occur if services are slow, not just when they are unavailable. Even a 400 ms increase in latency can reduce Google search traffic by 0.5% (Bansal et al., 2020). Traditional RCA rely on manual efforts and take a long time to identify the root cause. In contrast, GenAI-based solutions have shown better performance by reducing the mean time to detect (MTTD) and mean time to resolve (MTTR). These systems also suggest automated remediation that engineers can understand. This helps build trust and reduce financial impact (Sun et al., 2024).

Although traditional RCA are cost-effective, they have low accuracy and limited flexibility. They assume a simple linear relationship between variables, such as metrics, logs, or alerts. They used a fixed threshold to detect abnormal behavior. As a result, they struggle to adapt to dynamic environments (Fernando et al., 2025; Girish & Rao, 2023; J. Qiu et al., 2019). Empirical studies have shown the poor performance of statistical techniques, such as principal component analysis (PCA) or vector auto regression (VAR), with F1 scores between 0.4 and 0.5 (Z. Chen et al., 2022; Fernando et al., 2025; S. Jin et al., 2024). This means that alerts can be skipped or recurring incidents can be raised (Ge et al., 2024; Panwar et al., 2025; J. Qiu et al., 2019).

Classical ML uses algorithms such as support vector machines, decision trees, and isolation forests to detect issues in systems. It shows an accuracy of 90-95% in identifying issues in controlled settings. However, they require a labeled dataset, and the cost of labeling a dataset is very high. These models may suffer from high data imbalance, where normal pattern logs will appear most of the time, and error exceptions are one of the cases. Deep learning approaches further improve the performance using models such as autoencoders or LSTM (Fernando et al., 2025; C. Zhang, Peng, Sha, et al., 2022). This helps reduce the MTTD and failure impact, resulting in a measurable operational return on investment (ROI) (Hrusto et al., 2022; Jha et al., 2025).

Causal inference and Explainable AI (XAI) help improve interpretability and stability by providing trust and expert validation (L. Wang et al., 2023; Yu et al., 2023). For example, BALANCE is an attribution-based method that outperforms state-of-the-art models. This reduces the computational overhead and enables transparent reasoning (C. Chen et al., 2023). The LLM model provides a human-readable explanation. These models can automate reasoning, rule generation, and knowledge retrieval. Empirical evidence has shown a 45% reduction in incidents, which provides major operational benefits (S. Jin et al., 2024). Despite its benefits, GenAI incurs additional training costs, such as inference expenses, limited context windows, and hallucinations (Y. Chen et al., 2023; Zhang et al., 2024).

AIOPS delivers the highest ROI by reducing the mean time to resolve (MTTR) and labor dependency. Production deployment has observed RCA time reduction up to 96.5% and improvement in service availability and SLA compliance (Ding et al., 2023; W. Gu et al., 2025).

1.6.1 ROI Framework

Framework can be broken into three main component.

Quantifiable Losses. RCA framework can help in reducing financial loss due to downtime or service interruption. This can be measured using direct financial loss, prevention of SLA penalties and regulatory risk mitigation. For example, cloud providers such as AWS can give 10% service credit if up-time has dropped below 99.99% and 30% credit for up-time below 99.0% (Bansal et al., 2020; Gulenko, 2020). Monetary Authority of Singapore (MAS) has fined DBS Bank with six months pause on non essential business activities due to repeated failure (S. Jin et al., 2024). Table summarizes the financial loss impact due to slow RCA.

Table 7
Financial and Operational Impact of IT Service Disruptions

Metric	Financial Impact / Statistic	Source
Hourly Outage Cost	A single hour of downtime for Amazon on major shopping days can cost up to $100 million.	(Chakraborty et al., 2024)
Daily Outage Cost	A 24-hour incident affecting mission-critical microservices on AWS could result in $3.4 billion in revenue loss.	(Chakraborty et al., 2024)
Per-Minute Cost	The average cost of downtime for enterprise-level services exceeds $5,600 per minute.	(Chakraborty et al., 2024)

Metric	Financial Impact / Statistic	Source
Specific Incident Cost	A 37-minute downtime incident on YouTube was estimated to cost Google $1.7 million in advertising revenue alone.	(Chakraborty et al., 2024)
Latency Impact	A 400 ms increase in latency can cause a 0.5% drop in Google Search volume, directly affecting revenue.	(Bansal et al., 2020)
SLA Penalties	Cloud providers such as AWS offer 10% to 30% service credits when uptime drops below specified thresholds, such as 99.99% or 99.0%, respectively.	(Bansal et al., 2020)

Metric	Financial Impact / Statistic	Source
Regulatory Risk	Operational failures can lead to severe regulatory actions, such as the six-month pause on non-essential business activities imposed on DBS Bank by the Monetary Authority of Singapore.	(S. Jin et al., 2024)
Global Impact	IT downtime is estimated to cost $26.5 billion in lost revenue annually.	(C. Chen et al., 2023)

Operational Efficiency. Framework measure mean-time-to-resolve (MTTR) by drastically reducing the root cause identification. For example, Adobe has deployed the AI driven solution for root cause identification which reduced the overall analysis time from 3 hours to 15 minutes (Guntupalli, 2025). 41% SRE has reported that their work is manual and repetitive due to lack of automation (Hwang et al., 2021). AI driven RCA attempt to fix the root cause by fixing underlying configuration or code issues and prevent recurring failures (S. Jin et al., 2024).

Table 8
Operational Efficiency and Performance Improvements from AI-Driven Root Cause Analysis

Metric	Improvement / Efficiency Gain	Source
MTTR Reduction (Adobe)	Reduced Root Cause Analysis (RCA) time from 3 hours to less than 15 minutes.	(Guntupalli, 2025)
MTTR Reduction (Banking)	Reduced diagnosis time in a commercial bank from 30 minutes to 5 minutes.	(Chakraborty et al., 2024)
MTTR Reduction (GenAI)	The TrioXpert framework achieved diagnosis in an average of 26 seconds, compared with 2.5 hours for human experts.	(Sun, Luo, et al., 2025)
Investigation Savings	The DeCaf system saved engineers an average of 20 hours per investigation.	(Bansal et al., 2020)
Toil Reduction	Site Reliability Engineers (SREs) report that 41% to 50% of their work is manual “toil”; AI automation can help reclaim this time.	(Chakraborty et al., 2024)

Metric	Improvement / Efficiency Gain	Source
Incident Reduction	GenAI-driven RCA led to a 45% reduction in major incidents and a 45.5% reduction in change failure rates over one year in a global financial company.	(S. Jin et al., 2024)
Accuracy Improvement	AI-driven methods such as CloudRCA can help SREs reduce failure-resolution time by more than 20% compared with manual methods.	(Y. Zhang et al., 2021)
Actionable Guidance	GenAI copilots such as Nissist can achieve a 98.93% reduction in Time-to-Mitigate (TTM) for simple incidents by providing guidance to engineers.	(An et al., 2024)

Cost Of Implementation. Framework has to take into account continuous model training cost. If model's are mot trained regularly they may show drift the output (Guntupalli, 2025). Deep

learning model can be computationally expensive and resource hungry (Aktaş & Kilinc, 2024; Fernando et al., 2025). Additionally these models could be black box in nature. SRE may not be readily accepting its output. This cost vs marginal accuracy gains needs to be evaluated with trust factor and used judiciously (Chakraborty et al., 2024; Z. Chen et al., 2020).

GENERATIVE AI / LLM COST ANALYSIS – DETAILED.

Baseline Cost Calculation.

- Log processing: ~2,000 tokens (logs)
- Metric processing: ~1,000 tokens (time-series data)
- Trace analysis: ~2,000 tokens (request graph)
- Context/instructions: ~500 tokens
- **Total per attempt: ~5,500 tokens**

GPT-4 Pricing (as of 2025).

- Input: $0.03 per 1,000 tokens
- Output: $0.06 per 1,000 tokens
- **Cost per attempt:** (5.5 × $0.03) + (1.5 × $0.06) ≈ **$0.26**

Typical RCA Requires Multiple Attempts.

- Initial diagnosis attempt: 1 × $0.26
- Hypothesis refinement: 1–2 × $0.26 (if results are unclear)
- Verification queries: 1 × $0.26

- **Average attempts per incident: 3**
- **Cost per incident:** $3 \times \$0.26 \approx$ **$0.78**

Table 9
Annual Inference Cost by Incident Volume

Incidents per Month	Incidents per Year	Estimated Cost per Incident	Estimated Annual Inference Cost
10	120	$0.78	$93.60
20	240	$0.78	$187.20
100	1,200	$0.78	$936.00
500	6,000	$0.78	$4,680.00

Table 10
Cost and Resource Requirements of AI-Driven Root Cause Analysis

Cost Component	Details & Estimates	Source
Inference Cost (LLM)	Identifying a single fault type using a model such as GPT-4 can cost approximately $0.04 per input; this can become expensive at high log volumes.	(Han et al., 2024)

Cost Component	Details & Estimates	Source
Training Resources	Fine-tuning large models (e.g., GPT-3-scale models) is resource-intensive, requiring clusters such as 16 V100 32 GB GPUs for training and 4 V100 GPUs for inference.	(Zhang et al., 2024)
Data Labeling	Obtaining high-quality labeled data for supervised learning is "significantly tricky" and incurs substantial labor costs for expert annotation.	(Y. Chen et al., 2023; Cheng et al., 2023; Duan, Bao, et al., 2024)
MLOps Debt	Continuous retraining is required to prevent "model staleness" caused by system evolution and concept drift; failing to do so can make maintenance more expensive than the value provided.	(Guntupalli, 2025)

Cost Component	Details & Estimates	Source
Computational Overhead	Data preparation modules, such as those used in MicroRank, can consume up to 60% CPU utilization on a single core to mine detailed trace information.	(Yu et al., 2021)
Instrumentation Cost	Heavy instrumentation, such as kernel-level tracking required by some proactive models, can be impractical in large production systems because of performance overhead.	(Gan et al., 2021, 2022b)
Knowledge Base	Constructing the necessary Knowledge Graph to support GenAI agents requires end-to-end automation of the SDLC and represents a continuous, resource-intensive effort.	(S. Jin et al., 2024)

```
ROI = ((Revenue Loss Avoided + SLA Penalties + Labor Cost Reduction) -
(Compute cost + MLOps maintenance))/(Compute cost + MLOps maintenance)
```

Where:

- Revenue Loss Avoided = Downtime cost/hour × hours saved/year
- Service Level Agreement (SLA) Penalties Prevented = Annual SLA credit reductions
- Labor Cost Reduction = (Current SRE time – New SRE time) × SRE cost/hour
- Compute Cost = GPU/infrastructure annual expense
- MLOps Maintenance = Model retraining + data collection + monitoring
- Interpretation:
  - ROI > 100% = Positive return (benefits exceed costs)
  - ROI = 0% = Break-even (benefits equal costs)
  - ROI < 0% = Negative return (costs exceed benefits)

Example: ROI of 1,000% means for every $1 spent, you get $10 back. Over a year, a $50k investment with 1,000% ROI generates $500k in benefits. Below table summarizes the impact of AI based RCA from operational, cost and implementation perspective.

Table 11
Comparison of Traditional and AI-Driven Root Cause Analysis Approaches

Metric	Traditional/Manual RCA	Advanced AI/GenAI RCA
Resolution Time (MTTR)	Hours (2–3 hours average) (Guntupalli, 2025; Sun, Zheng, et al., 2025)	Minutes/seconds (<5 minutes) (W. Gu et al., 2025; Sarda et al., 2023)

Metric	Traditional/Manual RCA	Advanced AI/GenAI RCA
Cost of Downtime	High (e.g., $100M/hour exposure)	Minimized through rapid mitigation
Operational Cost	High (human labor/toil)	Lower through automation
Inference/Compute Cost	Negligible	High ($0.04/query, plus GPU costs)
Setup/Data Cost	Low	High (data labeling, Knowledge Graph construction)
Accuracy/Resolution	Coarse (service-level)	Fine-grained (code/metric-level)
Recurrence Rate	High (symptom-focused remediation)	Potentially lower through root-cause elimination (Guntupalli, 2025)

1.7 Gaps and Future

The literature has identified operational, technical, and economic gaps in the RCA systems. Lets explore them.

1.7.1 Technical and Data Challenges

Root cause localization and multi-modal fusion. Current RCA systems face challenges in analyzing data from different sources, such as metrics, logs, or traces, in sufficient depth. However, multi-modal data may be inconsistent or contain noise. The state of the system is not captured correctly owing to the incomplete handling of multi-modal data (Y. Chen et al., 2023; Fu et al., 2025).

The velocity of the observability data is very high, which creates challenges for both traditional and modern RCA systems (Wang et al., 2024; Yu et al., 2023). Although multi-modal data processing is widely accepted, it requires additional data fusion techniques (Sun, Luo, et al., 2025). Sometimes, one modality data can hide the symptoms of other modality data (Fu et al., 2025).

Traditional machine learning approaches have provided service-level results that lack code or configuration-level insights for accurate fault localization (Aktaş & Kilinc, 2024; Wang et al., 2024; Xin et al., 2024). Engineers must manually investigate the service to identify the fine-grained root cause (Yu et al., 2023; Z. Zhu et al., 2024).

Overall, achieving a high-quality integration of multi-modal data for precise root cause localization is challenging.

Data, organization, and MLOPS barriers. The lack of labeled dataset is a major challenge in AI-based RCA systems. Labeled faults require efforts from expert engineers, and real fault data are occasionally rare (Harutyunyan et al., 2023). This limits the ability of the supervised model to perform, and the unsupervised model performs with fewer labels, but it generates false positive data (C. Chen et al., 2023; Hardt et al., 2024; Y. Lin, n.d.; Soldani & Brogi, 2023; Sun,

Luo, et al., 2025). Organizations are reluctant to proceed with autonomous cloud deployments because of the lack of a standardization framework and limitations in cross-vendor usage (Ahmad et al., 2025; Shetty et al., 2024). Frequent updates may cause concept drift, requiring continuous training. This creates additional costs and infrastructure burdens (Shan et al., 2025; Xiang et al., 2025; Zhang et al., 2024). In-context learning (ICL) helps reduce training costs, but requires a robust foundational system (Ahmad et al., 2025; Zhang et al., 2024).

Interpretability and Explainability. Many RCA systems fail to gain the trust of service reliability engineers (SRE), who must perform remediation actions (Harutyunyan et al., 2023). These systems do not produce interpretable and actionable insights for SRE. Deep Learning and Machine Learning algorithms are black box for SRE. They do not provide clear reasoning or causal relationships behind remediation actions (C. Chen et al., 2023; Sun, Luo, et al., 2025). However, traditional RCA provide high-level outcomes, such as identifying failure services (Aktaş & Kilinc, 2024; Wang et al., 2024; Yu et al., 2023). These methods fail to identify the exact root cause. The SRE must perform additional steps to determine the root cause (Yu et al., 2023; Z. Zhu et al., 2024). This highlights the need for clear, actionable, and transparent explanations of failures.

Temporal causality and causal inference in dynamic state. Identifying the cause-and-effect relationship with respect to time is a challenge. Many ML/DL-based systems ignore the temporal relationship owing to computational cost or performance. Typically, these systems fail to capture event sequences. For example, the CPU usage of a service increases before an increase in the latency (Hou, 2025; C.-M. Lin et al., 2024; L. Wang et al., 2023). Additionally, multi-modal fusion ignores the time-based nature of fault propagation, which does not reflect the cause-and-

effect relationship in the causal graph (Hou, 2025). Causal discovery algorithms, such as the PC algorithm, are very slow and resource-intensive. In addition, their performance is poor when the data are limited (C. Chen et al., 2023; Pham et al., 2024c).

The problem of temporal dependencies exacerbates in large-scale micro-service environments, where dependencies are dynamically changed. These changes occur because of deployment, autoscaling of services, or configuration updates. Service dependencies and causal relationships can shift while an anomaly occurs (Fu et al., 2025; L. Wang et al., 2023). In such scenarios, statistical rule-based methods struggle to perform, but deep learning algorithms can still handle the situation; however, their learning and training costs are often high (Fu et al., 2025; W. Gu et al., 2025).

Metric-based techniques can produce misleading signals. Correlated signals are not necessarily causally related to each other. This creates false alarms and incorrect root cause identification (W. Gu et al., 2025; Hardt et al., 2024; Soldani & Brogi, 2023). Graph-related systems that create dependency graphs from static rules require manual efforts to define traces and rules. Creating a reliable causal graph from operational data is difficult (Gan, 2021; Pham et al., 2024c). The graph becomes unstable when false correlations build causal links under dynamic conditions. RCA must adapt to changing system topologies and identify how faults evolve over time. The high cost and complexity of building causal links between dependencies create challenges in fully autonomous remediation flows.

Model Hallucination and Reliability. A major challenge with GenAI-based RCA is model hallucination. The model can produce fabricated outputs, and acting on them may result in misinterpretation in the analysis (Cornacchia et al., 2025). LLM are effective in processing huge logs, but

they lack deterministic behavior,which makes their output unpredictable (Y. Gu et al., 2025; Pei et al., 2025). Data are often processed in chunks, and processing large volumes of data at once is not possible. This limits their usefulness in end-to-end RCA (Zhang et al., 2024). LLM actions are constrained by predefined tools that are costly (Shi et al., 2025). To reduce this, external guardrails and a human-in-the-loop approach are required. Fully autonomous remediation remains immature and unstable (Cheng et al., 2023; Esposito et al., 2025; Sarda et al., 2023).

1.7.2 Operational Gap

Simplistic or Isolated Models. Many existing RCA methods rely on overly simplistic anomaly detection techniques like N-Sigma or BIRCH clustering (Pham et al., 2024a; Wang et al., 2024). Studies often consider AD and RCA as different stages (Lee et al., 2023b; Wang et al., 2024). When these simple detectors provide incorrect results, such as noisy labels, they significantly reduce the success and accuracy of subsequent steps in identifying the root cause (Lee et al., 2023b; Pham et al., 2024a; Wang et al., 2024).

Although various studies have focused solely on identifying the root cause, they have assumed the existence of an anomaly detection module that can accurately detect failures and trigger RCA (Pham et al., 2024a). In practice, current RCA methods commonly use simple techniques such as N-sigma, even when the data are not normally distributed (Wang et al., 2024). Based on the accuracy of anomaly detection, further localization steps were performed (Wang et al., 2024).

This highlights the need for a unified end-to-end solution for RCA, such as BARO and KPIRoot+ (W. Gu et al., 2025; Pham et al., 2024a; Pham et al., 2024c).

1.7.3 Economic Gap

False Alarm and missed anomalies. False positives occur when the system classifies normal events as problems, and false negatives occur when the system fails to highlight the actual issues (Soldani & Brogi, 2023; S. Zhang et al., 2026). In the first case, resources and time are wasted on analyzing an issue that did not occur, and in the second case, failure to identify the actual issue leads to prolonged downtime. This also breaks the subsequent RCA pipeline flow, such as the localization and remediation steps (Wang et al., 2024). Both issues incur additional costs and have economic impacts.

Economic Impact of Advanced AI Models. GenAI and DL models require incremental training using the latest dataset. This adds to the training and resource consumption costs. In addition, the GenAI model can process a limited set of tokens, and multi-modal data may be sufficiently large for processing and generating accurate results. The cost of a single input for identifying one fault using GPT-4 is approximately $0.04 (Han et al., 2024). The economic viability of the GenAI model is constrained by a shift in development efforts to per-inference token consumption (Han et al., 2024; Shan et al., 2025). Data collection requires a high CPU, memory, and disk storage. In comparison, traditional RCA are slow but cost-effective. Organizations must be careful in choosing LLM models and comparing cost-benefit analyses. To address this gap, current research is focusing on in-context learning (ICL) (Zhang et al., 2024, 2024).

1.7.4 Research Gap

Fragmented AIOps System. Currently, anomaly detection (AD), root cause localization (RCL), and automated remediation (AR) are treated as separate steps. Currently, no end-to-end platform seamlessly connects RCA with automated execution. An automated solution is effective when anomaly detection and root cause localization are accurate and reliable. Fragmented RCA phases create challenges in building fully automated AIOps systems. In addition, the remediation engine should correctly interpret the insights and execute appropriate recovery actions (Cheng et al., 2023). According to (Sarda et al., 2023), future research should focus on integrated, self-contained AIOps systems.

Addressing root causes. The focus of SRE is to restore the service as quickly as possible and reduce damage. This leads to fixing the symptoms over the actual root cause, such as restarting the service or patching the software (S. Jin et al., 2024). This may leave the actual configuration, code, or design flaws unaddressed, which creates technical debt (S. Jin et al., 2024; Zhang et al., 2024). Using GenAI with the Five Why technique can help in finding the actual root cause. This may reveal hidden issues from the code and help prevent the same problem from occurring again (S. Jin et al., 2024).

Benchmark and continuous learning are also important aspects of this field. There is a lack of publicly available dataset to identify the root cause. It is difficult for researchers to compare different RCA methods and produce experimental results. Some simulated dataset fail to capture the variability and complexity of actual production systems (Cheng et al., 2023; S. Zhang et al.,

2026). Another issue is the dynamic system change or concept drift. The system evolves over time, and static anomaly detection and RCA models quickly lose accuracy owing to the lack of training (Soldani & Brogi, 2023). Most RCA approaches do not utilize human feedback and knowledge to improve the model's performance (Cheng et al., 2023; C. Wang et al., 2024). This leads to repeated errors and declining reliability. In addition, there is no widely accepted benchmark to evaluate the RCA performance (Cheng et al., 2023). The AIOps lab provides a holistic evaluation framework with support for fine-grained root cause analysis and comprehensive assessment of agents (Cheng et al., 2023). The lack of standard benchmark and real-world dataset prevents the fair and reliable evaluation of RCA systems.

Human-in-the-loop Optimization. Most RCA systems do not use human knowledge to improve model accuracy via a human feedback loop. Human factors, such as organizational and team structures, are under explored (Cheng et al., 2023; L. Wang et al., 2023). These dependencies can significantly affect system reliability. For example, the T-CausalRCA framework introduced human expertise via hierarchical reinforcement learning with human feedback (HRLHF) to support causal discovery (L. Wang et al., 2023). The SRE has domain expertise and a good understanding of causal relationships and system behavior. Effective AIOps should support human-in-the-loop feedback for better performance. Agentic AI can assist in service decoupling strategies to reduce failure propagation (Esposito et al., 2025). Future research should focus on developing methods to quantify, capture, and apply human expertise (Esposito et al., 2025; L. Wang et al., 2023; S. Zhang et al., 2026).

Model Adaptability. Future research should focus on adaptable models. When dynamic system changes occur, the model should be adapted to new types of failures. This will help move away from dynamic, rigid models. Model retraining incurs additional costs and resources (Pei et al., 2025). Research increasingly highlights few-shot and meta-learning techniques. This helps the model learn and adapt to new changes without any concept drift. This also reduces the need for large dataset and heavy computational resources (Duan, Bao, et al., 2024). The RCA system should be able to highlight the failure, even if no prior information is available on the failure (Zhang et al., 2024). Future studies should prioritize adaptive learning methods (Duan, Bao, et al., 2024).

References

Adeba, J. L., Kim Deuk-Hun, & Kwak, J. (2025). Temporal Decay Loss for Adaptive Log Anomaly Detection in Cloud Environments. *Sensors*, *25*(9), 2649. Publicly Available Content Database (3203224519). https://doi.org/10.3390/s25092649

Ahmad, A., Li, P., Piechocki, R., & Inacio, R. (2025). Anomaly Detection in Offshore Open Radio Access Network Using Long Short-Term Memory Models on a Novel Artificial Intelligence-Driven Cloud-Native Data Platform. *Engineering Applications of Artificial Intelligence*, *161*, 112274. https://doi.org/10.1016/j.engappai.2025.112274

Aktaş, K., & Kilinc, H. H. (2024). Interaction Prediction and Anomaly Detection in a Microservices-based Telecommunication Platform. *Proceedings of the 2024 International Conference on Software and Systems Processes*, 56–65. https://doi.org/10.1145/3666015.3666017

Almajed, H., Alsaqer, A., & Albuali, A. (2025). Towards Effective Anomaly Detection: Machine Learning Solutions in Cloud Computing. *International Journal of Advanced Computer Science and Applications*, *16*(2). https://doi.org/10.14569/IJACSA.2025.01602132

An, K., Yang, F., Lu, J., Li, L., Ren, Z., Huang, H., Wang, L., Zhao, P., Kang, Y., Ding, H., Lin, Q., Rajmohan, S., Zhang, D., & Zhang, Q. (2024). *Nissist: An Incident Mitigation Copilot based on Troubleshooting Guides* (arXiv:2402.17531). arXiv. https://doi.org/10.48550/arXiv.2402.17531

Araujo, I., Antunes, N., & Vieira, M. (2023). Intrusion Detection and Tolerance for Microservice Applications. *12th Latin-American Symposium on Dependable and Secure Computing*, 176–181. https://doi.org/10.1145/3615366.3622794

Bansal, C., Renganathan, S., Asudani, A., Midy, O., & Janakiraman, M. (2020). DeCaf: diagnosing and triaging performance issues in large-scale cloud services. *Proceedings of the ACM/IEEE 42nd International Conference on Software Engineering: Software Engineering in Practice*, 201–210. https://doi.org/10.1145/3377813.3381353

Boesel, F., Schläpfer, L., Pozidis, H., & Gusat, M. (2023). Multivariate Anomaly Detection with Domain Clustering. *Proceedings of the 2023 ACM Symposium on Cloud Computing*, 193–199. https://doi.org/10.1145/3620678.3624655

C. Gao, L. Shi, H. Sun, T. Ma, Y. Chen, Q. Pei, & Y. Chen. (1 Aug.1, 2025). An Efficient Anomaly Detection Model Based on Tensor Decomposition and VARIMA for High-Dimensional Multivariate Time Series. *IEEE Internet of Things Journal*, *12*(15), 29443–29459. https://doi.org/10.1109/JIOT.2025.3570876

Chakraborty, S., Agarwal, S., Garg, S., Sethia, A., Pandey, U. N., Aggarwal, V., & Saini, S. (2023). *ESRO: Experience Assisted Service Reliability against Outages* (arXiv:2309.07230). arXiv. https://doi.org/10.48550/arXiv.2309.07230

Chakraborty, S., Agarwal, S., Garg, S., Sethia, A., Pandey, U. N., Aggarwal, V., & Saini, S. (2024). ESRO: Experience Assisted Service Reliability against Outages. *Proceedings of the 38th IEEE/ACM International Conference on Automated Software Engineering, ASE '23*, 255–267. https://doi.org/10.1109/ASE56229.2023.00131

Chang, B. R., Tsai, H.-F., & Chen, G.-R. (2024). Self-Adaptive Server Anomaly Detection Using Ensemble Meta-Reinforcement Learning. *Electronics*, *13*(12), 2348. https://doi.org/10.3390/electronics13122348

Chen, C., Yu, H., Lei, Z., Li, J., Ren, S., Zhang, T., Hu, S., Wang, J., & Shi, W. (2023). BALANCE: Bayesian Linear Attribution for Root Cause Localization. *Proceedings of the ACM on Management of Data*, *1*(1), 1–26. https://doi.org/10.1145/3588949

Chen, F., Zhang, Y., Fan, L., Liang, Y., Pang, G., Wen, Q., & Deng, S. (2024). Cluster-Wide Task Slowdown Detection in Cloud System. *Proceedings of the 30th ACM SIGKDD Conference on Knowledge Discovery and Data Mining*, 266–277. https://doi.org/10.1145/3637528.3671936

Chen, F., zhang, Y., Qin, Z., Fan, L., Jiang, R., Liang, Y., Wen, Q., & Deng, S. (2024). *Learning Multi-Pattern Normalities in the Frequency Domain for Efficient Time Series Anomaly Detection* (arXiv:2311.16191). arXiv. https://doi.org/10.48550/arXiv.2311.16191

Chen, J., Zhang, R., Chen, P., Ren, J., Wu, Z., Wang, Y., Li, X., & Xiong, L. (2024). MTG_CD: Multi-scale learnable transformation graph for fault classification and diagnosis in microservices. *Journal of Cloud Computing*, *13*(1), 103. https://doi.org/10.1186/s13677-024-00666-0

Chen, N., Tu, H., Duan, X., Hu, L., & Guo, C. (2023). Semisupervised anomaly detection of multivariate time series based on a variational autoencoder. *Applied Intelligence*, *53*(5), 6074–6098. ABI/INFORM Collection (2778132167). https://doi.org/10.1007/s10489-022-03829-1

Chen, Y., Shetty, M., Somashekar, G., Ma, M., Simmhan, Y., Mace, J., Bansal, C., Wang, R., & Rajmohan, S. (2025). *AIOpsLab: A Holistic Framework to Evaluate AI Agents for Enabling Autonomous Clouds* (arXiv:2501.06706). arXiv. https://doi.org/10.48550/arXiv.2501.06706

Chen, Y., Xie, H., Ma, M., Kang, Y., Gao, X., Shi, L., Cao, Y., Gao, X., Fan, H., Wen, M., Zeng, J., Ghosh, S., Zhang, X., Zhang, C., Lin, Q., Rajmohan, S., Zhang, D., & Xu, T. (2023). *Automatic Root Cause Analysis via Large Language Models for Cloud Incidents* (arXiv:2305.15778). arXiv. https://doi.org/10.48550/arXiv.2305.15778

Chen, Z., Kang, Y., Li, L., Zhang, X., Zhang, H., Xu, H., Zhou, Y., Yang, L., Sun, J., Xu, Z., Dang, Y., Gao, F., Zhao, P., Qiao, B., Lin, Q., Zhang, D., & Lyu, M. R. (2020). Towards intelligent incident management: why we need it and how we make it. *Proceedings of the 28th ACM Joint Meeting on European Software Engineering Conference and Symposium on the Foundations of Software Engineering*, 1487–1497. https://doi.org/10.1145/3368089.3417055

Chen, Z., Liu, J., Gu, W., Su, Y., & Lyu, M. R. (2022). *Experience Report: Deep Learning-based System Log Analysis for Anomaly Detection* (arXiv:2107.05908). arXiv. https://doi.org/10.48550/arXiv.2107.05908

Cheng, Q., Sahoo, D., Saha, A., Yang, W., Liu, C., Woo, G., Singh, M., Saverese, S., & Hoi, S. C. H. (2023). *AI for IT Operations (AIOps) on Cloud Platforms: Reviews, Opportunities and Challenges* (arXiv:2304.04661). arXiv. https://doi.org/10.48550/arXiv.2304.04661

Cornacchia, A., Alabdulaal, I., Saghier, I., Mirdad, A., Fayoumi, O., & Canini, M. (2025). Between Promise and Pain: The Reality of Automating Failure Analysis in Microservices with LLMs. *Proceedings of the 16th ACM SIGOPS Asia-Pacific Workshop on Systems*, 155–167. https://doi.org/10.1145/3725783.3764388

D'Angelo, A., & d'Aloisio, G. (2024). Grammar-Based Anomaly Detection of Microservice Systems Execution Traces. *Companion of the 15th ACM/SPEC International Conference on Performance Engineering*, 77–81. https://doi.org/10.1145/3629527.3651844

Ding, R., Zhang, C., Wang, L., Xu, Y., Ma, M., Wu, X., Zhang, M., Chen, Q., Gao, X., Gao, X., Fan, H., Rajmohan, S., Lin, Q., & Zhang, D. (2023). TraceDiag: Adaptive, Interpretable, and Efficient Root Cause Analysis on Large-Scale Microservice Systems. *Proceedings of the 31st ACM Joint European Software Engineering Conference and Symposium on the Foundations of Software Engineering*, 1762–1773. https://doi.org/10.1145/3611643.3613864

Duan, Y., Bao, H., Bai, G., Wei, Y., Xue, K., You, Z., Zhang, Y., Liu, B., Chen, J., Wang, S., & Ou, Z. (2024). Learning to Diagnose: Meta-Learning for Efficient Adaptation in Few-Shot AIOps Scenarios. *Electronics*, *13*(11), 2102. https://doi.org/10.3390/electronics13112102

Duan, Y., Xue, K., & Sun, H. (n.d.). *LogEDL: Log Anomaly Detection via Evidential Deep Learning*.

Duan, Y., Xue, K., Sun, H., Bao, H., Wei, Y., You, Z., Zhang, Y., Jiang, X., Yang, S., Chen, J., Duan, B., & Ou, Z. (2024). LogEDL: Log Anomaly Detection via Evidential Deep Learning. *Applied Sciences*, *14*(16), 7055. Publicly Available Content Database (3097822159). https://doi.org/10.3390/app14167055

Esposito, M., Bakhtin, A., Ahmad, N., Robredo, M., Su, R., Lenarduzzi, V., & Taibi, D. (2025). *Autonomic Microservice Management via Agentic AI and MAPE-K Integration* (arXiv:2506.22185). arXiv. https://doi.org/10.48550/arXiv.2506.22185

Fernando, D., Rodriguez, M. A., Arroba, P., Ismail, L., & Buyya, R. (2025). Efficient Training Approaches for Performance Anomaly Detection Models in Edge Computing Environments. *ACM Transactions on Autonomous and Adaptive Systems*, *20*(2), 1–27. https://doi.org/10.1145/3725736

Fu, N., Cheng, G., Teng, Y., Dai, G., Yu, S., & Chen, Z. (2025). Intelligent Root Cause Localization in MicroService Systems: A Survey and New Perspectives. *ACM Computing Surveys*, *57*(12), 1–37. https://doi.org/10.1145/3736755

Gan, Y. (2021). Designing and Managing Large-Scale Interactive Microservices in Datacenters [Ph.D., Cornell University]. In *ProQuest Dissertations and Theses*. ProQuest Dissertations & Theses Global (2622279536). https://www.proquest.com/dissertations-theses/designing-managing-large-scale-interactive/docview/2622279536/se-2?accountid=25283

Gan, Y., Liang, M., Dev, S., Lo, D., & Delimitrou, C. (2021). *Sage: Using Unsupervised Learning for Scalable Performance Debugging in Microservices* (arXiv:2101.00267). arXiv. https://doi.org/10.48550/arXiv.2101.00267

Gan, Y., Liang, M., Dev, S., Lo, D., & Delimitrou, C. (2022a). Enabling Practical Cloud Performance Debugging with Unsupervised Learning. *ACM SIGOPS Operating Systems Review*, *56*(1), 34–41. https://doi.org/10.1145/3544497.3544503

Gan, Y., Liang, M., Dev, S., Lo, D., & Delimitrou, C. (2022b). Practical and Scalable ML-Driven Cloud Performance Debugging With Sage. *IEEE Micro*, *42*(4), 27–36. https://doi.org/10.1109/MM.2022.3169445

Gan, Y., Liu, G., Zhang, X., Zhou, Q., Wu, J., & Jiang, J. (2023). Sleuth: A Trace-Based Root Cause Analysis System for Large-Scale Microservices with Graph Neural Networks. *Proceedings of the 28th ACM International Conference on Architectural Support for Programming Languages and Operating Systems, Volume 4*, 324–337. https://doi.org/10.1145/3623278.3624758

Gao, H., Xin, R., Chen, P., Li, X., Lu, N., & You, P. (2025). Memory-augment graph transformer based unsupervised detection model for identifying performance anoma-

lies in highly-dynamic cloud environments. *Journal of Cloud Computing*, *14*(1), 40. https://doi.org/10.1186/s13677-025-00766-5

Ge, H., Ji, X., Peng, F., Chen, R., Xiang, N., Zhang, K., & Wu, W. (2024). SRdetector: Sequence Reconstruction Method for Microservice Anomaly Detection. *Electronics*, *14*(1), 65. https://doi.org/10.3390/electronics14010065

Girish, L., & Rao, S. K. N. (2023). Anomaly detection in cloud environment using artificial intelligence techniques. *Computing*, *105*(3), 675–688. https://doi.org/10.1007/s00607-021-00941-x

Gu, W., Zhong, R., Yu, G., Sun, X., Liu, J., Huo, Y., Chen, Z., Zhang, J., Gu, J., Yang, Y., & Lyu, M. R. (2025). *KPIRoot+: An Efficient Integrated Framework for Anomaly Detection and Root Cause Analysis in Large-Scale Cloud Systems* (arXiv:2506.04569). arXiv. https://doi.org/10.48550/arXiv.2506.04569

Gu, Y., Xiong, Y., Mace, J., Jiang, Y., Hu, Y., Kasikci, B., & Cheng, P. (2025). *Argos: Agentic Time-Series Anomaly Detection with Autonomous Rule Generation via Large Language Models* (arXiv:2501.14170). arXiv. https://doi.org/10.48550/arXiv.2501.14170

Gulenko, A. (2020). *Autonomic self-healing in cloud computing platforms*.

Guntupalli, R. (2025). *AI-driven anomaly detection and root cause analysis: Using machine learning on logs, metrics, and traces to detect subtle performance anomalies, security threats, or failures in complex cloud environments*. https://doi.org/https://doi.org/10.30574/wjarr.2025.26.2.1521

Gupta, L. (2019). Management and Security of Multi-Cloud Applications [Ph.D., Washington University in St. Louis]. In *ProQuest Dissertations and Theses*. ProQuest Dissertations & The-

ses Global (2212217278). https://www.proquest.com/dissertations-theses/management-security-multi-cloud-applications/docview/2212217278/se-2?accountid=25283

Hagemann, T., & Katsarou, K. (2020a). A Systematic Review on Anomaly Detection for Cloud Computing Environments. *2020 3rd Artificial Intelligence and Cloud Computing Conference*, 83–96. https://doi.org/10.1145/3442536.3442550

Hagemann, T., & Katsarou, K. (2020b). Reconstruction-based anomaly detection for the cloud: A comparison on the Yahoo! Webscope S5 dataset. *Proceedings of the 2020 4th International Conference on Cloud and Big Data Computing*, 68–75. https://doi.org/10.1145/3416921.3416934

Han, Y., Du, Q., Huang, Y., Wu, J., Tian, F., & He, C. (2024). The Potential of One-Shot Failure Root Cause Analysis: Collaboration of the Large Language Model and Small Classifier. *Proceedings of the 39th IEEE/ACM International Conference on Automated Software Engineering*, 931–943. https://doi.org/10.1145/3691620.3695475

Hardt, M., Orchard, W. R., Blöbaum, P., Kasiviswanathan, S., & Kirschbaum, E. (2024). *The PetShop Dataset -- Finding Causes of Performance Issues across Microservices* (arXiv:2311.04806). arXiv. https://doi.org/10.48550/arXiv.2311.04806

Harutyunyan, A., Poghosyan, A., Harutyunyan, L., Aghajanyan, N., Bunarjyan, T., & Vinck, A. J. H. (2023). Challenges and Experiences in Designing Interpretable KPI-diagnostics for Cloud Applications. *JUCS - Journal of Universal Computer Science*, *29*(11), 1298–1318. https://doi.org/10.3897/jucs.112570

He, H., Li, X., Chen, P., Chen, J., Liu, M., & Wu, L. (2024). Efficiently localizing system anomalies for cloud infrastructures: a novel Dynamic Graph Transformer based Parallel Framework. *Journal of Cloud Computing*, *13*(1), 115. https://doi.org/10.1186/s13677-024-00677-x

He, Z., Hu, G., & Lee, R. B. (2023). CloudShield: Real-time Anomaly Detection in the Cloud. *Proceedings of the Thirteenth ACM Conference on Data and Application Security and Privacy*, 91–102. https://doi.org/10.1145/3577923.3583639

Heeb, Z., Kalinagac, O., Soussi, W., & Gür, G. (2023). IoMiRCA: Root cause analysis in IoT-extended 5G microservice environments. *Proceedings of the 38th ACM/SIGAPP Symposium on Applied Computing*, 106–108. https://doi.org/10.1145/3555776.3577840

Hou, J. (2025). *Research on fault diagnosis and root cause analysis based on full stack observability* (arXiv:2509.12231). arXiv. https://doi.org/10.48550/arXiv.2509.12231

Hrusto, A., Engström, E., & Runeson, P. (2022). Optimization of anomaly detection in a microservice system through continuous feedback from development. *Proceedings of the 10th IEEE/ACM International Workshop on Software Engineering for Systems-of-Systems and Software Ecosystems*, 13–20. https://doi.org/10.1145/3528229.3529382

Hu, J., Long, L., Sui, H., Gu, Z., & Zheng, G. (2025). CFTL: System Log Parsing Method Driven from Clustering According to First Token and Length for Anomaly Detection. *Applied Sciences*, *15*(4), 1740. https://doi.org/10.3390/app15041740

Huang, H., Zhang, X., Chen, P., He, Z., Chen, Z., Yu, G., Chen, H., & Sun, C. (2024). TraStrainer: Adaptive Sampling for Distributed Traces with System Runtime State. *Proceedings of the ACM on Software Engineering*, *1*(FSE), 473–493. https://doi.org/10.1145/3643748

Huang, J., Chen, P., Yu, G., Wang, Y., Huang, H., & He, Z. (2024). *FaaSRCA: Full Lifecycle Root Cause Analysis for Serverless Applications* (arXiv:2412.02239). arXiv. https://doi.org/10.48550/arXiv.2412.02239

Huang, J., Yang, Y., Yu, H., Li, J., & Zheng, X. (2023). *Twin Graph-based Anomaly Detection via Attentive Multi-Modal Learning for Microservice System* (arXiv:2310.04701). arXiv. https://doi.org/10.48550/arXiv.2310.04701

Huang, T., Chen, P., & Li, R. (2022). A Semi-Supervised VAE Based Active Anomaly Detection Framework in Multivariate Time Series for Online Systems. *Proceedings of the ACM Web Conference 2022*, 1797–1806. https://doi.org/10.1145/3485447.3511984

Hwang, J., Shwartz, L., Wang, Q., Batta, R., Kumar, H., & Nidd, M. (2021). *FIXME: Enhance Software Reliability with Hybrid Approaches in Cloud* (arXiv:2102.09336). arXiv. https://doi.org/10.48550/arXiv.2102.09336

Islam, M. S., Rakha, M. S., Pourmajidi, W., Sivaloganathan, J., Steinbacher, J., & Miranskyy, A. (2025). Anomaly Detection in Large-Scale Cloud Systems: An Industry Case and Dataset. *2025 IEEE/ACM 47th International Conference on Software Engineering: Software Engineering in Practice (ICSE-SEIP)*, 377–388. https://doi.org/10.1109/ICSE-SEIP66354.2025.00039

Jha, N., Lin, S., Jayaraman, S., Frohling, K., Constantinides, C., & Patel, D. (2025). *LLM Assisted Anomaly Detection Service for Site Reliability Engineers: Enhancing Cloud Infrastructure Resilience* (arXiv:2501.16744). arXiv. https://doi.org/10.48550/arXiv.2501.16744

Ji, C., & Luo, H. (2025a). *Cloud-Based AI Systems: Leveraging Large Language Models for Intelligent Fault Detection and Autonomous Self-Healing* (arXiv:2505.11743). arXiv. https://doi.org/10.48550/arXiv.2505.11743

Ji, C., & Luo, H. (2025b). *Leveraging Large Language Model for Intelligent Log Processing and Autonomous Debugging in Cloud AI Platforms* (arXiv:2506.17900). arXiv. https://doi.org/10.48550/arXiv.2506.17900

Ji, X., Zhang, L., Zhang, W., Peng, F., Mao, Y., Liao, X., & Zhang, K. (2025). LEMAD: LLM-Empowered Multi-Agent System for Anomaly Detection in Power Grid Services. *Electronics*, *14*(15), 3008. https://doi.org/10.3390/electronics14153008

Jian, H. (n.d.). *Research on fault diagnosis and root cause analysis based on full stack observability*.

Jiang, G. (2024). Artificial intelligence-based adaptive anomaly detection technology for IaaS cloud virtual machines. *Journal of Engineering and Applied Science*, *71*(1), 102. https://doi.org/10.1186/s44147-024-00439-1

Jin, S., & Bei, Z. (n.d.). *Breaking the Cycle of Recurring Failures: Applying Generative AI to Root Cause Analysis in Legacy Banking Systems*.

Jin, S., Bei, Z., Chen, B., & Xia, Y. (2024). *Breaking the Cycle of Recurring Failures: Applying Generative AI to Root Cause Analysis in Legacy Banking Systems* (arXiv:2411.13017). arXiv. https://doi.org/10.48550/arXiv.2411.13017

Jin, Y., Yang, Z., Liu, J., & Xu, X. (2025). *Anomaly Detection and Early Warning Mechanism for Intelligent Monitoring Systems in Multi-Cloud Environments Based on LLM* (arXiv:2506.07407). arXiv. https://doi.org/10.48550/arXiv.2506.07407

Jonnaikode, S. (n.d.). *Detecting Software Anomalies in Microservices Using Spectrograms, Scalograms, and Convolutional Neural Networks*.

Jonnaikode, S. (2025). Detecting Software Anomalies in Microservices using Spectrograms, Scalograms, and Convolutional Neural Networks [M.S., California Polytechnic State University]. In *ProQuest Dissertations and Theses*. ProQuest Dissertations & Theses Global (3235008860). https://www.proquest.com/dissertations-theses/detecting-software-anomalies-microservices-using/docview/3235008860/se-2?accountid=25283

Khodabandeh, G., Ezaz, A., Babaei, M., & Ezzati-Jivan, N. (2025). *Utilizing Graph Neural Networks for Effective Link Prediction in Microservice Architectures* (arXiv:2501.15019). arXiv. https://doi.org/10.48550/arXiv.2501.15019

L. Wu, J. Tordsson, E. Elmroth, & O. Kao. (2020). MicroRCA: Root Cause Localization of Performance Issues in Microservices. *NOMS 2020 - 2020 IEEE/IFIP Network Operations and Management Symposium*, 1–9. https://doi.org/10.1109/NOMS47738.2020.9110353

Lee, C., Yang, T., Chen, Z., Su, Y., & Lyu, M. R. (2023a). *Eadro: An End-to-End Troubleshooting Framework for Microservices on Multi-source Data* (arXiv:2302.05092). arXiv. https://doi.org/10.48550/arXiv.2302.05092

Lee, C., Yang, T., Chen, Z., Su, Y., & Lyu, M. R. (2023b). Eadro: An End-to-End Troubleshooting Framework for Microservices on Multi-Source Data. *Proceedings of the 45th International Conference on Software Engineering, ICSE '23*, 1750–1762. https://doi.org/10.1109/ICSE48619.2023.00150

Lin, C.-M., Chang, C., Wang, W.-Y., Wang, K.-D., & Peng, W.-C. (2024). *Root Cause Analysis In Microservice Using Neural Granger Causal Discovery* (arXiv:2402.01140). arXiv. https://doi.org/10.48550/arXiv.2402.01140

Lin, Y. (n.d.). *Self-Supervised Distributed Machine Learning for Robust Containerized Systems*.

Liu, D., He, C., Peng, X., Lin, F., Zhang, C., Gong, S., Li, Z., Ou, J., & Wu, Z. (2021). MicroHECL: High-Efficient Root Cause Localization in Large-Scale Microservice Systems. *2021 IEEE/ACM 43rd International Conference on Software Engineering: Software Engineering in Practice (ICSE-SEIP)*, 338–347. https://doi.org/10.1109/ICSE-SEIP52600.2021.00043

Liu, Y., Pei, C., Xu, L., Chen, B., Sun, M., Zhang, Z., Sun, Y., Zhang, S., Wang, K., Zhang, H., Li, J., Xie, G., Wen, X., Nie, X., Ma, M., & Pei, D. (2025). OpsEval: A Comprehensive Benchmark Suite for Evaluating Large Language Models' Capability in IT Operations Domain. *Proceedings of the 33rd ACM International Conference on the Foundations of Software Engineering*, 503–513. https://doi.org/10.1145/3696630.3728572

Ma, M., Xu, J., Wang, Y., Chen, P., Zhang, Z., & Wang, P. (2020). AutoMAP: Diagnose Your Microservice-based Web Applications Automatically. *Proceedings of The Web Conference 2020*, 246–258. https://doi.org/10.1145/3366423.3380111

Nikiforov, R. (2018). *Clustering-based Anomaly Detection for microservices* (arXiv:1810.02762). arXiv. https://doi.org/10.48550/arXiv.1810.02762

Nobre, J., Pires, E. J. S., & Reis, A. (2023a). Anomaly Detection in Microservice-Based Systems. *Applied Sciences*, *13*(13), 7891. https://doi.org/10.3390/app13137891

Nobre, J., Pires, E. J. S., & Reis, A. (2023b). Anomaly Detection in Microservice-Based Systems. *Applied Sciences*, *13*(13), 7891. https://doi.org/10.3390/app13137891

Noor, A. I. (n.d.). *Real-Time QoS Monitoring and Anomaly Detection on Microservice-based Applications in Cloud-Edge Infrastructure*.

Panwar, A., Pal, H., Chen, J., Cho, K., Jiang, R., Zhao, M., & Krishnamurthy, R. (2025). *Reasoning-based Anomaly Detection Framework: A Real-time, Scalable, and Automated Approach to Anomaly Detection Across Domains* (arXiv:2510.03486). arXiv. https://doi.org/10.48550/arXiv.2510.03486

Pei, C., Wang, Z., Liu, F., Li, Z., Liu, Y., He, X., Kang, R., Zhang, T., Chen, J., Li, J., Xie, G., & Pei, D. (2025). *Flow-of-Action: SOP Enhanced LLM-Based Multi-Agent System for Root Cause Analysis* (arXiv:2502.08224). arXiv. https://doi.org/10.48550/arXiv.2502.08224

Petrov, V., Gennadinik, A., & Avksentieva, E. (2022). Metrics for machine learning evaluation methods in cloud monitoring systems. *2022 8th International Conference on Computer Technology Applications*, 168–175. https://doi.org/10.1145/3543712.3543715

Pham, L., Ha, H., & Zhang, H. (2024a). BARO: Robust Root Cause Analysis for Microservices via Multivariate Bayesian Online Change Point Detection. *Proceedings of the ACM on Software Engineering*, *1*(FSE), 2214–2237. https://doi.org/10.1145/3660805

Pham, L., Ha, H., & Zhang, H. (2024b). *BARO: Robust Root Cause Analysis for Microservices via Multivariate Bayesian Online Change Point Detection*. *1*(FSE), 2214–2237. https://doi.org/10.1145/3660805

Pham, L., Ha, H., & Zhang, H. (2024c). *Root Cause Analysis for Microservice System based on Causal Inference: How Far Are We?* (arXiv:2408.13729). arXiv. https://doi.org/10.48550/arXiv.2408.13729

Poghosyan, A., Harutyunyan, A., Grigoryan, N., & Pang, C. (2023). Distributed Tracing for Troubleshooting of Native Cloud Applications via Rule-Induction Systems. *JUCS - Journal of Universal Computer Science*, *29*(11), 1274–1297. https://doi.org/10.3897/jucs.112513

Qi, Z., Xie, H., Li, Y., Tan, J., Li, F., & Lui, J. C. S. (2022). *LPC-AD: Fast and Accurate Multivariate Time Series Anomaly Detection via Latent Predictive Coding* (arXiv:2205.08362). arXiv. https://doi.org/10.48550/arXiv.2205.08362

Qiu, H., Banerjee, S. S., Jha, S., Kalbarczyk, Z. T., & Iyer, R. K. (2020). *FIRM: An Intelligent Fine-Grained Resource Management Framework for SLO-Oriented Microservices* (arXiv:2008.08509). arXiv. https://doi.org/10.48550/arXiv.2008.08509

Qiu, J., Du, Q., & Qian, C. (2019). KPI-TSAD: A Time-Series Anomaly Detector for KPI Monitoring in Cloud Applications. *Symmetry*, *11*(11), 1350. https://doi.org/10.3390/sym11111350

Rouf, R., Rasolroveicy, M., Litoiu, M., Nagar, S., Mohapatra, P., Gupta, P., & Watts, I. (2024). InstantOps: A Joint Approach to System Failure Prediction and Root Cause Identification in Microserivces Cloud-Native Applications. *Proceedings of the 15th ACM/SPEC International Conference on Performance Engineering*, 119–129. https://doi.org/10.1145/3629526.3645047

Sarda, K., Namrud, Z., & Rouf, R. (2023). *ADARMA: Auto-Detection and Auto-Remediation of Microservice Anomalies by Leveraging Large Language Models*.

Sarika, P. K., Badampudi, D., Josyula, S. P., & Usman, M. (2023). Automating Microservices Test Failure Analysis using Kubernetes Cluster Logs. *Proceedings of the 27th International Conference on Evaluation and Assessment in Software Engineering*, 192–195. https://doi.org/10.1145/3593434.3593472

Shan, A., Kaur, J., Singh, R., Banka, T., Yavatkar, R., & Sridhar, T. (2025). *RCA Copilot: Transforming Network Data into Actionable Insights via Large Language Models* (arXiv:2507.03224). arXiv. https://doi.org/10.48550/arXiv.2507.03224

Shetty, M., Bansal, C., Kumar, S., Rao, N., & Nagappan, N. (2021). *SoftNER: Mining Knowledge Graphs From Cloud Incidents* (arXiv:2101.05961). arXiv. https://doi.org/10.48550/arXiv.2101.05961

Shetty, M., Bansal, C., Kumar, S., Rao, N., Nagappan, N., & Zimmermann, T. (2021). *Neural Knowledge Extraction From Cloud Service Incidents* (arXiv:2007.05505). arXiv. https://doi.org/10.48550/arXiv.2007.05505

Shetty, M., Chen, Y., Somashekar, G., Ma, M., Simmhan, Y., Zhang, X., Mace, J., Vandevoorde, D., Las-Casas, P., Gupta, S. M., Nath, S., Bansal, C., & Rajmohan, S. (2024). *Building*

AI Agents for Autonomous Clouds: Challenges and Design Principles (arXiv:2407.12165). arXiv. https://doi.org/10.48550/arXiv.2407.12165

Shi, B., Luo, Y., Wang, J., Zhao, Y., Zhang, S., Hao, B., Zhao, C., Sun, Y., Zhang, Z., Sun, R., Li, H., Song, W., Chen, X., Miao, J., & Pei, D. (2025). FlowXpert: Expertizing Troubleshooting Workflow Orchestration with Knowledge Base and Multi-Agent Coevolution. *Proceedings of the 31st ACM SIGKDD Conference on Knowledge Discovery and Data Mining V.2*, 4839–4850. https://doi.org/10.1145/3711896.3737221

Shivakumar, M. (n.d.). *AIOps–Driven Adaptive Anomaly Detection in Evolving Cloud Environments Using Transfer Learning*.

Shivakumar, M. (2025). Aiops–driven Adaptive Anomaly Detection in Evolving Cloud Environments using Transfer Learning [M.S., California Polytechnic State University]. In *ProQuest Dissertations and Theses*. ProQuest Dissertations & Theses Global (3235005709). https://www.proquest.com/dissertations-theses/aiops-driven-adaptive-anomaly-detection-evolving/docview/3235005709/se-2?accountid=25283

Soldani, J., & Brogi, A. (2023). Anomaly Detection and Failure Root Cause Analysis in (Micro) Service-Based Cloud Applications: A Survey. *ACM Computing Surveys*, *55*(3), 1–39. https://doi.org/10.1145/3501297

Somashekar, G., Dutt, A., Adak, M., Lorido Botran, T., & Gandhi, A. (2024). GAMMA: Graph Neural Network-Based Multi-Bottleneck Localization for Microservices Applications. *Proceedings of the ACM Web Conference 2024*, 3085–3095. https://doi.org/10.1145/3589334.3645665

Sun, Y., Lin, Z., Shi, B., Zhang, S., Ma, S., Jin, P., Zhong, Z., Pan, L., Guo, Y., & Pei, D. (2025). Interpretable Failure Localization for Microservice Systems Based on Graph Autoencoder.

ACM Transactions on Software Engineering and Methodology, *34*(2), 1–28. https://doi.org/10.1145/3695999

Sun, Y., Luo, Y., Wen, X., Yuan, Y., Nie, X., Zhang, S., Liu, T., & Luo, X. (2025). *TrioXpert: An Automated Incident Management Framework for Microservice System* (arXiv:2506.10043). arXiv. https://doi.org/10.48550/arXiv.2506.10043

Sun, Y., Shi, B., Mao, M., Ma, M., Xia, S., Zhang, S., & Pei, D. (2024). ART: A Unified Unsupervised Framework for Incident Management in Microservice Systems. *Proceedings of the 39th IEEE/ACM International Conference on Automated Software Engineering*, 1183–1194. https://doi.org/10.1145/3691620.3695495

Sun, Y., Zheng, T., Wen, X., Kuang, W., Liu, H., Zhang, S., Shen, C., Wu, B., & Pei, D. (2025). A Multimodal Intelligent Change Assessment Framework for Microservice Systems Based on Large Language Models. *Proceedings of the 33rd ACM International Conference on the Foundations of Software Engineering*, 378–388. https://doi.org/10.1145/3696630.3728561

Tang, P., Tang, S., Pu, H., Miao, Z., & Wang, Z. (2025). *MicroRCA-Agent: Microservice Root Cause Analysis Method Based on Large Language Model Agents* (arXiv:2509.15635). arXiv. https://doi.org/10.48550/arXiv.2509.15635

Tao, L., Zhang, S., Jia, Z., Sun, J., Ma, M., Li, Z., Sun, Y., Yang, C., Zhang, Y., & Pei, D. (2024). Giving Every Modality a Voice in Microservice Failure Diagnosis via Multimodal Adaptive Optimization. *Proceedings of the 39th IEEE/ACM International Conference on Automated Software Engineering*, 1107–1119. https://doi.org/10.1145/3691620.3695489

Tien, C., Huang, T., Tien, C., Huang, T., & Kuo, S. (2019). KubAnomaly: Anomaly detection for the Docker orchestration platform with neural network approaches. *Engineering Reports*, *1*(5), e12080. https://doi.org/10.1002/eng2.12080

Vervaet, A. (2023). *MoniLog: An Automated Log-Based Anomaly Detection System for Cloud Computing Infrastructures* (arXiv:2304.11940). arXiv. https://doi.org/10.48550/arXiv.2304.11940

Wang, C., Huang, T., Li, M., Chen, P., & Chen, Z. (2024). A Bayesian LSTM Based Active Anomaly Detection Service for Large Online Systems. *Proceedings of the 15th Asia-Pacific Symposium on Internetware*, 407–416. https://doi.org/10.1145/3671016.3674818

Wang, H., Wu, Z., Jiang, H., Huang, Y., Wang, J., Kopru, S., & Xie, T. (2022). Groot: an event-graph-based approach for root cause analysis in industrial settings. *Proceedings of the 36th IEEE/ACM International Conference on Automated Software Engineering, ASE '21*, 419–429. https://doi.org/10.1109/ASE51524.2021.9678708

Wang, L., Zhang, C., Ding, R., Xu, Y., Chen, Q., Zou, W., Chen, Q., Zhang, M., Gao, X., Fan, H., Rajmohan, S., Lin, Q., & Zhang, D. (2023). Root Cause Analysis for Microservice Systems via Hierarchical Reinforcement Learning from Human Feedback. *Proceedings of the 29th ACM SIGKDD Conference on Knowledge Discovery and Data Mining*, 5116–5125. https://doi.org/10.1145/3580305.3599934

Wang, S., Sandeepa, C., Senevirathna, T., Siniarski, B., Nguyen, M.-D., Marchal, S., & Liyanage, M. (2024). *Towards Accountable and Resilient AI-Assisted Networks: Case Studies and Future Challenges*. https://doi.org/10.1109/eucnc/6gsummit60053.2024.10597060

Wang, T., & Qi, G. (2024). *A Comprehensive Survey on Root Cause Analysis in (Micro) Services: Methodologies, Challenges, and Trends* (arXiv:2408.00803). arXiv. https://doi.org/10.48550/arXiv.2408.00803

Wang, T., Qi, G., & Wu, T. (2024). *KGroot: Enhancing Root Cause Analysis through Knowledge Graphs and Graph Convolutional Neural Networks* (arXiv:2402.13264). arXiv. https://doi.org/10.48550/arXiv.2402.13264

Wang, Y., Zhu, Z., Fu, Q., Ma, Y., & He, P. (2024). MRCA: Metric-level Root Cause Analysis for Microservices via Multi-Modal Data. *Proceedings of the 39th IEEE/ACM International Conference on Automated Software Engineering*, 1057–1068. https://doi.org/10.1145/3691620.3695485

Wang, Y., Mäntylä, M. V., Demeyer, S., Beyazit, M., Kisaakye, J., & Nyyssölä, J. (2025). Cross-System Categorization of Abnormal Traces in Microservice-Based Systems via Meta-Learning. *Proceedings of the ACM on Software Engineering*, *2*(FSE), 576–598. https://doi.org/10.1145/3715742

Xiang, Y., Chen, C. P., Zeng, L., Yin, W., Liu, X., Li, H., & Xu, W. (2025). *Simplifying Root Cause Analysis in Kubernetes with StateGraph and LLM* (arXiv:2506.02490). arXiv. https://doi.org/10.48550/arXiv.2506.02490

Xie, S., Wang, J., He, H., Wang, Z., Zhao, Y., Zhang, N., & Li, B. (2026). TVDiag: A Task-oriented and View-invariant Failure Diagnosis Framework for Microservice-based Systems with Multimodal Data. *ACM Transactions on Software Engineering and Methodology*, *35*(2), 1–39. https://doi.org/10.1145/3734868

Xie, Z., Zhang, S., Geng, Y., Zhang, Y., Ma, M., Nie, X., Yao, Z., Xu, L., Sun, Y., Li, W., & Pei, D. (2024). Microservice Root Cause Analysis With Limited Observability Through Intervention Recognition in the Latent Space. *Proceedings of the 30th ACM SIGKDD Conference on Knowledge Discovery and Data Mining*, 6049–6060. https://doi.org/10.1145/3637528.3671530

Xin, R., Liu, H., Chen, P., Grosso, P., & Zhao, Z. (2024). FIRED: a fine-grained robust performance diagnosis framework for cloud applications. *Future Generation Computer Systems*, *155*, 300–311. https://doi.org/10.1016/j.future.2024.02.014

Xing, S., Wang, Y., & Liu, W. (2025). Multi-Dimensional Anomaly Detection and Fault Localization in Microservice Architectures: A Dual-Channel Deep Learning Approach with Causal Inference for Intelligent Sensing. *Sensors*, *25*(11), 3396. https://doi.org/10.3390/s25113396

Yao, Z., Pei, C., Chen, W., Wang, H., Su, L., Jiang, H., Xie, Z., Nie, X., & Pei, D. (2024). Chain-of-Event: Interpretable Root Cause Analysis for Microservices through Automatically Learning Weighted Event Causal Graph. *Companion Proceedings of the 32nd ACM International Conference on the Foundations of Software Engineering*, 50–61. https://doi.org/10.1145/3663529.3663827

Yu, G., Chen, P., Chen, H., Guan, Z., Huang, Z., Jing, L., Weng, T., Sun, X., & Li, X. (2021). MicroRank: End-to-End Latency Issue Localization with Extended Spectrum Analysis in Microservice Environments. *Proceedings of the Web Conference 2021*, 3087–3098. https://doi.org/10.1145/3442381.3449905

Yu, G., Chen, P., Li, Y., Chen, H., Li, X., & Zheng, Z. (2023). Nezha: Interpretable Fine-Grained Root Causes Analysis for Microservices on Multi-modal Observability Data. *Proceedings of the 31st ACM Joint European Software Engineering Conference and Symposium on the Foundations of Software Engineering*, 553–565. https://doi.org/10.1145/3611643.3616249

Zhang, C., Dong, Z., Peng, X., Zhang, B., & Chen, M. (2024). Trace-based Multi-Dimensional Root Cause Localization of Performance Issues in Microservice Systems. *Proceedings of*

the IEEE/ACM 46th International Conference on Software Engineering, 1–12. https://doi.org/10.1145/3597503.3639088

Zhang, C., Peng, X., Sha, C., Zhang, K., Fu, Z., Wu, X., Lin, Q., & Zhang, D. (2022). DeepTraLog: trace-log combined microservice anomaly detection through graph-based deep learning. *Proceedings of the 44th International Conference on Software Engineering*, 623–634. https://doi.org/10.1145/3510003.3510180

Zhang, C., Peng, X., Zhou, T., Sha, C., Yan, Z., Chen, Y., & Yang, H. (2022). TraceCRL: contrastive representation learning for microservice trace analysis. *Proceedings of the 30th ACM Joint European Software Engineering Conference and Symposium on the Foundations of Software Engineering*, 1221–1232. https://doi.org/10.1145/3540250.3549146

Zhang, S., Jin, P., Lin, Z., Sun, Y., Zhang, B., Xia, S., Li, Z., Zhong, Z., Ma, M., Jin, W., Zhang, D., Zhu, Z., & Pei, D. (2023). *Robust Failure Diagnosis of Microservice System through Multimodal Data* (arXiv:2302.10512). arXiv. https://doi.org/10.48550/arXiv.2302.10512

Zhang, S., Xia, S., Fan, W., Shi, B., Xiong, X., Zhong, Z., Ma, M., Sun, Y., & Pei, D. (2026). Failure Diagnosis in Microservice Systems: A Comprehensive Survey and Analysis. *ACM Transactions on Software Engineering and Methodology*, *35*(1), 1–55. https://doi.org/10.1145/3715005

Zhang, S., Zhu, J., Hao, B., Sun, Y., Nie, X., Zhu, J., Liu, X., Li, X., Ma, Y., & Pei, D. (2024). Fault Diagnosis for Test Alarms in Microservices through Multi-source Data. *Companion Proceedings of the 32nd ACM International Conference on the Foundations of Software Engineering*, 115–125. https://doi.org/10.1145/3663529.3663833

Zhang, W., Guo, H., Yang, J., Tian, Z., Zhang, Y., Yan, C., Li, Z., Li, T., Shi, X., Zheng, L., & Zhang, B. (2024). *mABC: multi-Agent Blockchain-Inspired Collaboration for root cause analysis*

in micro-services architecture (arXiv:2404.12135). arXiv. https://doi.org/10.48550/arXiv.2404.12135

Zhang, X., Wang, Q., Li, M., Yuan, Y., Xiao, M., Zhuang, F., & Yu, D. (2025). *TAMO: Fine-Grained Root Cause Analysis via Tool-Assisted LLM Agent with Multi-Modality Observation Data in Cloud-Native Systems* (arXiv:2504.20462). arXiv. https://doi.org/10.48550/arXiv.2504.20462

Zhang, X., Ghosh, S., Bansal, C., Wang, R., Ma, M., Kang, Y., & Rajmohan, S. (2024). *Automated Root Causing of Cloud Incidents using In-Context Learning with GPT-4* (arXiv:2401.13810). arXiv. https://doi.org/10.48550/arXiv.2401.13810

Zhang, Y., Guan, Z., Qian, H., Xu, L., Liu, H., Wen, Q., Sun, L., Jiang, J., Fan, L., & Ke, M. (2021). CloudRCA: A Root Cause Analysis Framework for Cloud Computing Platforms. *Proceedings of the 30th ACM International Conference on Information & Knowledge Management*, 4373–4382. https://doi.org/10.1145/3459637.3481903

Zhao, Z., Wang, Z., Zhang, T., Shen, Z., Dong, H., Lei, Z., Ma, X., Xu, G., Ding, Z., & Yang, Y. (2025). *CHASE: A Causal Hypergraph based Framework for Root Cause Analysis in Multimodal Microservice Systems* (arXiv:2406.19711). arXiv. https://doi.org/10.48550/arXiv.2406.19711

Zheng, L., Chen, Z., & Chen, H. (2025). *Online Multi-modal Root Cause Identification in Microservice Systems* (arXiv:2410.10021). arXiv. https://doi.org/10.48550/arXiv.2410.10021

Zheng, L., Chen, Z., He, J., & Chen, H. (2024). MULAN: Multi-modal Causal Structure Learning and Root Cause Analysis for Microservice Systems. *Proceedings of the ACM Web Conference 2024*, 4107–4116. https://doi.org/10.1145/3589334.3645442

Zhou, X., Peng, X., Xie, T., Sun, J., Ji, C., Liu, D., Xiang, Q., & He, C. (2019). Latent error prediction and fault localization for microservice applications by learning from system trace logs. *Proceedings of the 2019 27th ACM Joint Meeting on European Software Engineering Conference and Symposium on the Foundations of Software Engineering*, 683–694. https://doi.org/10.1145/3338906.3338961

Zhu, Y., Wang, J., Li, B., Tang, X., Li, H., Zhang, N., & Zhao, Y. (2024). *Root Cause Localization for Microservice Systems in Cloud-edge Collaborative Environments* (arXiv:2406.13604). arXiv. https://doi.org/10.48550/arXiv.2406.13604

Zhu, Z., Lee, C., Tang, X., & He, P. (2024). HeMiRCA: Fine-Grained Root Cause Analysis for Microservices with Heterogeneous Data Sources. *ACM Transactions on Software Engineering and Methodology*, *33*(8), 1–25. https://doi.org/10.1145/3674726